

허니팟 기반 사이버 공격 분석 시스템

공격자를 유인해 수집한 영문 보안 로그를, 규칙 기반 분류와 한국어 LLM 해설을
거쳐 비전문 관리자가 바로 읽고 대응할 수 있는 위협 보고로 바꾸는 통합 분석 파
이프라인.

판	기간	기준일
v2.6	2026-04 ~ 2026-09	2026-09-15
기반	분류기	해설 모델
T-Pot 24.04.1 (Telekom Security)	규칙 + ML 교차검증	EXAONE 3.5 7.8B

V2.6 에서 달라진 것

검증 규모가 시연 표본 1,736건에서 실제 공격 로그 1,026,903건으로 바뀌었다. AWS 에 띄웠던 T-Pot 이 10일간 받은 로그를 전부 색인해 파이프라인을 완주시켰다(13-1). 그 데이터로 분류기를 재학습해 **교차검증 모델을 v4 → v5 로 교체**하고 전량 재분류했으며, 합의율은 **81.5% → 99.9%** 가 됐다(11-7, 11-8).

v2.1 이 "18.5% 오류는 단일 학습 공백 하나" 라고 진단한 것은 **맞았다**. 실 문서로 학습하니 그 공백이 사라졌다. 다만 그 과정에서 **평가 방법 자체의 버그**를 찾았다 — 홀드아웃을 학습과 다른 인코더로 만들어 성능을 7%p 낮게 재고 있었다(11-9). 숫자보다 그 경위를 남긴다.

LLM 처리량도 실측으로 바뀌었다. 건당 27초는 3B 기준이고 현행 7.8B 는 **건당 약 60초**다(12-4).

그리고 그 31.8일은 사라졌다. 고위험 45,075건이 실제로는 **74개 공격 패턴**이고 상위 2개(둘 다 포트 445)가 89.8% 를 덮는다. 같은 이야기를 4만 번 쓰고 있었던 것이다. 패턴당 한 번만 해설하도록 고쳐 **31.8일 → 75분**이 됐고, 상위 3개만 해설하면 **3분에 90%** 가 덮인다(12-5). 게이트가 성립 조건이라는 결론은 그대로다 — 게이트 없이 100만 건은 여전히 불가능하다.

v2.6 은 9-8 의 마지막 과제였던 **온디맨드 추론 정책**을 매듭지은 판이다(9-9). "GPU 로 올리거나 외부 API 로 돌리거나 유료 등급에만 열거나" 셋 중 고르는 문제로 봤는데 **넷째 답이 있었다** — 고위험 사건은 이미 해설을 갖고 있어서 **대부분의 클릭에 추론이 필요 없다**. 실측 12/12 적중 · 평균 42ms(추론은 60초).

v2.5 는 **SaaS 형태로 굴릴 수 있는가**를 확인하고 막는 것을 건어낸 판이다(9-8). 결론부터: 브라우저 쪽은 처음부터 허니팟과 무관해 **이미 그 모양**이고, 막던 것은 **무인증 추론 엔드포인트와 직렬화되는 동시성**이었다. 자산 목록도 계정 단위로 옮겼다.

v2.4 는 그 뒤에 **화면**을 맞춘 판이다. 파이프라인만 패턴 단위로 가고 화면은 건별로 남아 같은 문단을 수백 줄 반복하고 있었다(9-5). 그리고 14장이 남겨둔 네 간극 중 **"고객 체감 긴급성"**에 손을 댔다 — 열어둔 포트를 대면 허니팟 데이터가 비로소 **내 얘기**가 된다(9-6). 판별 이력은 부록 A 에 있다.

목차

- | | |
|------------------------|---------------------------|
| 1. 허니팟이란 · 프로젝트 목표 | 8. 조용한 실패 |
| 2. 배경과 문제 정의 | 9. 2차 개발 — 사용자 인터페이스 |
| 3. 진행 경과와 방향 전환 | 10. 2차 개발 — 백엔드 결함 일곱 건 |
| 4. 시스템 아키텍처 | 11. ML 활성화 — 라벨러 개선과 교차검증 |
| 5. 기술 스택과 구현 | 12. LLM 모델 선정 |
| 6. 핵심 기술 과제 1 — ML 분류기 | 13. 검증 결과 |
| 7. 핵심 기술 과제 2 — LLM 판정 | 14. 상업화 가능성 |

15. 한계와 개선 방향

16. 결론

17. 부록 A – 판별 이력

01 허니팟이란 · 프로젝트 목표

1-1. 허니팟(Honeypot)이란

일부러 취약하게 열어둔 미끼 시스템이다. 실제 서비스가 아니므로 여기에 들어오는 접속은 정 의상 전부 비정상이고, 그래서 오탐이 거의 없는 공격 정보를 얻을 수 있다.

방화벽이나 백신은 "정상 트래픽 속에서 공격을 골라내는" 문제를 푼다. 정상과 공격이 섞여 있으니 오탐과 미 탐이 늘 따라온다. 허니팟은 문제를 뒤집는다. **아무도 정상적으로 접속할 이유가 없는 시스템**을 인터넷에 노출 해 두면, 들어오는 모든 접속이 곧 공격 시도다.

	일반 보안 장비	허니팟
목적	공격을 막는다	공격을 관찰·기록 한다
트래픽	정상 + 공격 혼재	사실상 전부 공격
오탐	구조적으로 발생	거의 없음
얻는 것	차단 로그	공격 수법·도구·출처·행동 전체

예를 들어 SSH 허니팟 `cowrie` 는 가짜 셸을 제공한다. 공격자가 로그인에 성공했다고 믿고 `wget` 으로 악 성코드를 내려받거나 리버스셸을 시도하면, **그 명령어가 그대로 기록된다**. 실제 시스템은 전혀 위험하지 않으 면서 공격자의 수법을 통째로 확보하는 것이다.

왜 중요한가

허니팟 데이터는 **실제로 지금 우리 IP 대역을 노리는 공격**의 기록이다. 어느 나라에서, 어떤 계정 사전으로, 어떤 악성코드를 심으려 하는지가 그대로 남는다. 위협 인텔리전스의 1차 자료로 쓰이는 이유다.

1-2. 그런데 기록만으로는 쓸 수 없다

문제는 그 다음이다. 허니팟이 남기는 것은 이런 형태다.

```
{ "eventid": "cowrie.login.failed", "username": "root",  
  "password": "123456", "src_ip": "203.0.113.10",  
  "dst_port": 22, "protocol": "ssh" }  
  
{ "alert.category": "Attempted Administrator Privilege Gain",  
  "alert.signature_id": 2010935, "payload_printable": "..." }
```

이것을 읽고 "무슨 일이 일어났고 무엇을 해야 하는가"로 옮기려면 허니팟별 이벤트 스키마와 MITRE ATT&CK 체계를 이미 알고 있어야 한다. 수집은 자동화됐지만 해석은 여전히 전문가 몫이라는 뜻이다.

1-3. 프로젝트 목표

허니팟이 모은 영문 원시 로그를, 보안 비전문가인 시스템 관리자가 그대로 읽고 바로 조치할 수 있는 한국어 위협 보고로 자동 변환한다.

#	목표	구체적 산출
1	자동 분류	모든 이벤트에 공격 유형 라벨(정찰·무차별대입·악성코드·침입)과 MITRE 위협 점수 0-100 부여
2	한국어 해설	고위험 이벤트에 대해 무슨 일인지 요약 + 지금 할 조치를 한국어 문장으로 생성
3	한눈에 보는 화면	한국어 대시보드, 실시간 공격 지도, 기간별 PDF 리포트
4	기존 자산 보존	T-Pot 본체를 수정하지 않는 사이드카 방식 — 업스트림 업데이트와 충돌 없음

1-4. 이 시스템으로 할 수 있는 것

완성된 파이프라인은 다음 질문들에 한국어로 답한다.

사용자	물어볼 것	시스템이 주는 것
시스템 관리자	어젯밤 우리 서버에 무슨 일이 있었나	24시간 요약 — 총 이벤트 · 공격 분류 · 고위험 건수, 한국어 요약문
시스템 관리자	이 공격, 지금 뭘 해야 하나	<code>solution_ko</code> — "해당 IP를 차단하고 SSH 계정 비밀번호를 변경하십시오" 같은 실행 가능한 조치
보안 담당자	어떤 유형 공격이 늘고 있나	유형별·허니팟별 추이, MITRE 기법 분포, 시간대별 히스토그램
보안 담당자	어디서 들어오나	실시간 공격 지도 — 출발지 국가에서 센서로 향하는 공격 아크
관리 책임자	보고서가 필요하다	기간 선택 → 한국어 PDF 리포트, CSV 내보내기

중소 규모 조직의 위협 조기 경보. 전담 보안 인력이 없는 조직이 자사 네트워크 대역에 허니팟을 하나 띄워 두면, 실제로 자기 대역을 노리는 공격이 무엇인지 한국어로 통보받는다. 대학·연구실 네트워크, 사내 인프라 모니터링, 보안 교육용 실습 환경이 직접적인 적용 대상이다.

02 배경과 문제 정의

1장에서 본 문제를 T-Pot 이라는 구체적인 플랫폼 위에 놓고 다시 정의한다. 무엇이 이미 해결돼 있고 무엇이 비어 있는지를 층위별로 가른다.

2-1. T-Pot 이 주는 것과 주지 않는 것

T-Pot 은 Telekom Security 가 공개한 허니팟 통합 배포판으로, 25종 내외의 허니팟과 Suricata·p0f 같은 패시브 센서를 하나의 Docker 스택으로 묶고 수집 로그를 Elasticsearch 에 적재한다. 인프라로서는 완성도가 높다.

그러나 운영자가 마주하는 화면은 **영문 Kibana 대시보드**이고, 그 안의 항목은 `cowrie.login.failed`, `alert.category: Attempted Administrator Privilege Gain` 같은 원시 이벤트다. 이것을 읽고 "무슨 일이 일어났고 무엇을 해야 하는가"로 옮기려면 허니팟별 이벤트 스키마와 MITRE ATT&CK 체계를 이미 알고 있어야 한다.

2-2. 문제

층위	T-POT 기본	필요한 것
수집	25종 허니팟 → ES 적재 충분	—
분류	없음. 원시 이벤트 그대로	공격 유형 라벨 + 위협 점수
해석	없음	한국어 요약 + 실행 가능한 대응 방안
제시	영문 Kibana	한국어 대시보드, 권한 분리, 리포트

즉 수집은 이미 해결됐고, 분류·해석·제시가 비어 있다. 이 셋을 채우는 것이 본 프로젝트의 범위다.

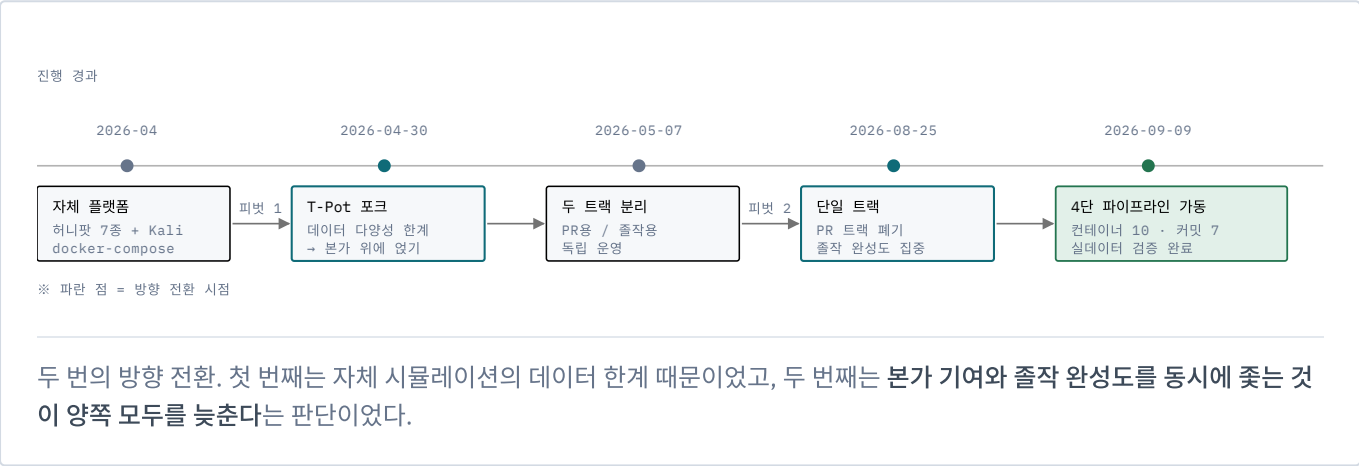
2-3. 설계 원칙

- **사이드카 방식** — T-Pot 본체(허니팟·Logstash·Kibana)를 수정하지 않는다. ES 를 읽고 자기 인덱스에만 쓰는 컨테이너를 덧붙인다. 업스트림 업데이트와 충돌하지 않는다.

- **단계별 인덱스 분리** — logstash-* (원본) → ml-analysis-* (분류) → llm-analysis-* (해설). 각 단계가 독립적으로 재실행·검증 가능하다.
- **실패 시 축소, 중단 아님** — 모델이 없으면 규칙으로, LLM 이 죽으면 결정론적 요약으로 내려간다. 문서가 조용히 사라지지 않는다.

03 진행 경과와 방향 전환

이 프로젝트는 두 번 방향을 크게 틀었다. 두 번 다 측정 결과가 계획을 이겼다.



3-1. 피벗 1 – 자체 플랫폼에서 T-Pot 포크로 (2026-04-30)

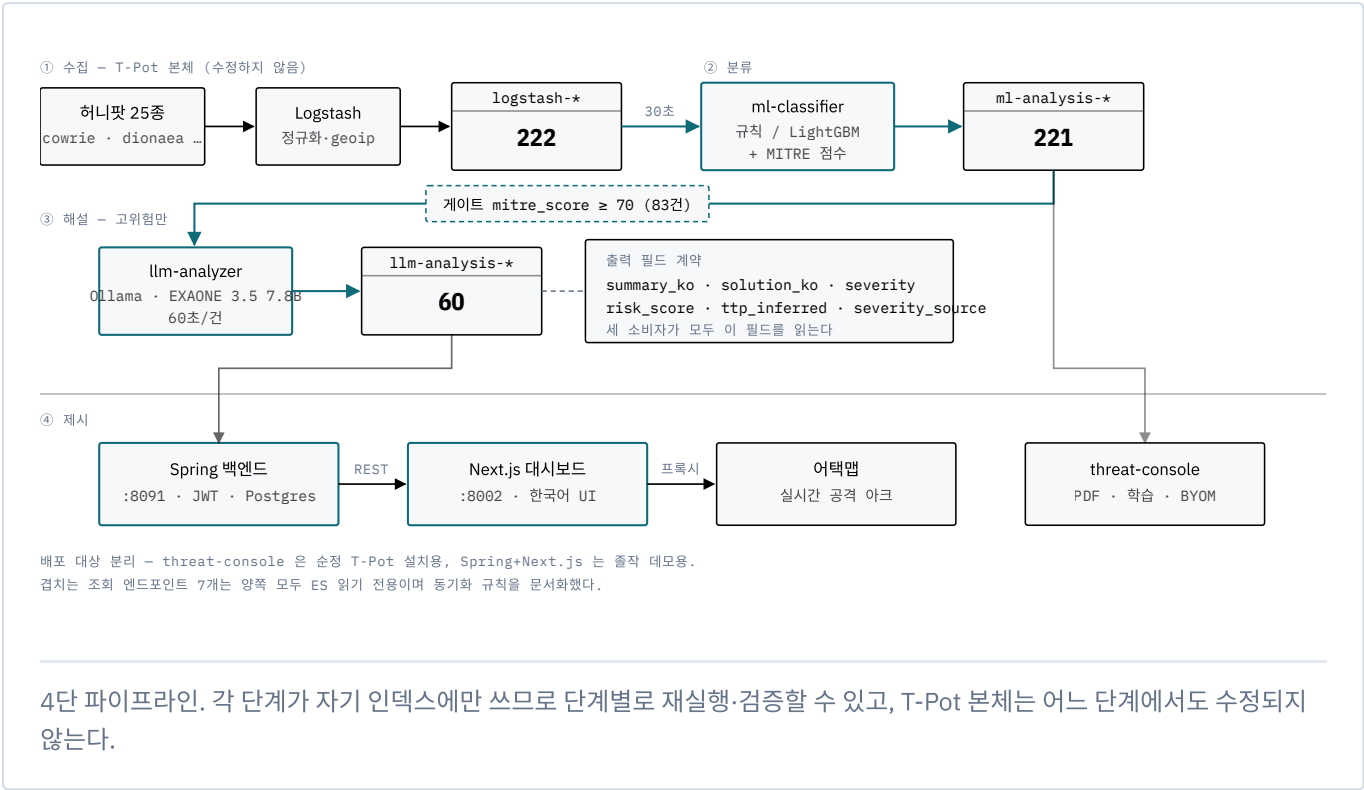
초기 구상은 허니팟 7종과 공격자 컨테이너(Kali)를 직접 묶은 자체 플랫폼이었다. 구현은 됐으나 **공격 데이터가 우리가 스크립트로 만든 것뿐**이라 분류 모델을 학습시킬 다양성이 나오지 않았다. 실제 인터넷 공격을 받는 T-Pot 위에 모듈을 얹는 방식으로 전환했다.

3-2. 피벗 2 – 단일 트랙으로 (2026-08-25)

한동안 본가(telekom-security/tpotce)에 PR 을 제출하는 트랙과 졸업작품 시연 트랙을 나눠 운영했다. PR 트랙은 본가 컨벤션(영어 전용, 머신 의존성 0, Spring/Next.js 금지)을 따라야 해서 **같은 기능을 두 벌로 유지**해야 했다.

본가 기여를 목표에서 제외하고 capstone/ 단일 트랙으로 통합했다. 삭제한 PR 트랙은 2,270 파일이었고, 그중 졸작 트랙에 없는 고유 파일은 2개뿐이었다 (사전 보존 후 삭제).

04 시스템 아키텍처



4-1. 실행 스택

컨테이너	포트	역할
capstone-es	19200	Elasticsearch 8.13.2 — 3개 인덱스 보유
capstone-postgres	5433	사용자·프로젝트·공격로그 영속
capstone-backend	8090	Spring Boot — JWT 인증 + ES 조회 6종
capstone-frontend	8001	Next.js 15 — 한국어 대시보드
capstone-ml-classifier	—	배치 분류 데몬 (30초 주기)
capstone-llm-analyzer	—	배치 해설 데몬 (60초 주기)
capstone-ollama	11434	로컬 LLM 추론
map_web / map_data / map_redis	64299	실시간 공격 지도

`docker compose up -d` 한 번으로 전체가 기동한다. ES 를 스택 안에 두고 `depends_on:`
`service_healthy` 로 기동 순서를 보장해, 이전에 백엔드가 의존성보다 먼저 떠서 반복 재시작하던 문제를

제거했다.

05 구현 상세

5-1. 코드 규모

ML - CLASSIFIER 줄 1,963 Python	LLM - ANALYZER 줄 (신규) 939 Python	THREAT - CONSOLE 줄 678 Python	SPRING 백엔드 1,597 Java · 45 파일
NEXT.JS 프론트 2,616 TS/TSX · 16 파일			

5-2. 기술 스택과 선택 이유

각 도구가 파이프라인의 어느 자리를 맡는지, 왜 그것을 골랐는지 정리한다.

수집 · 저장 계층 – 이미 있는 것을 쓴다

도구	하는 일	왜 이것인가
T-Pot 24.04.1	허니팟 25종을 하나의 Docker 스택으로 묶은 배포판. Telekom Security 가 공개·유지보수한다	허니팟을 하나씩 붙이면 각각 설정·네트워크·로그 형식이 달라 통합에만 프로젝트가 끝난다. T-Pot 은 그 통합을 이미 끝내놓았다 – 우리는 그 위에 얹기만 하면 된다
Elasticsearch 8.13	수집된 로그를 저장하고 검색·집계하는 문서 데이터베이스	로그처럼 스키마가 제각각인 문서를 그대로 넣고 나중에 집계할 수 있다. <code>terms</code> · <code>histogram</code> 집계 하나로 대시보드 차트가 나온다. T-Pot 이 이미 여기에 쌓고 있어 별도 적재가 필요 없다
Logstash	허니팟별로 제각각인 로그를 공통 필드로 정규화하고, IP 를 좌표로 바꿔(geoip) 붙인다	T-Pot 기본 구성. 우리가 손대지 않아도 <code>src_ip</code> · <code>dest_port</code> 같은 공통 필드와 지리 정보가 채워진 채로 들어온다

분석 계층 – 비어 있어서 우리가 만든 부분

도구	하는 일	왜 이것인가
규칙 라벨러 rule_label.py	허니팟 종류·이벤트 ID·명령 패턴을 보고 결정론적으로 공격 유형을 판정한다	ML 의 학습 정답 을 만들면서 동시에 풀백 이 된다. 모델이 없거나 신뢰도가 낮으면 자동으로 여기로 내려온다. 현재 운영 중인 분류기이기도 하다
LightGBM	공격 유형 다중분류 모델. 16개 특징(포트·프로토콜·로그인 시도 횟수·명령 패턴 등)으로 라벨을 예측	표 형태 데이터에서는 딥러닝보다 빠르고 정확한 경우가 많다. 학습이 초 단위라 재실험이 쉽고 모델 파일이 3MB 라 컨테이너에 그대로 넣을 수 있다. 규칙을 대체하지는 못하지만, 규칙과 교차검증하는 역할로 운영에 붙어 있다(11장)
MITRE ATT&CK	공격 기법을 T1110 (무차별 대입), T1046 (네트워크 스캔) 같은 표준 ID 로 표기하고 위협 점수 0-100 을 매긴다	전 세계 보안 업계가 쓰는 표준 분류 체계다. 자체 점수를 만들면 근거를 설명할 수 없지만, MITRE 기법 ID 는 외부 문서·타 보안 도구와 그대로 연결된다
Ollama EXAONE 3.5 7.8B	로컬에서 LLM 을 구동하는 런타임. 고위험 이벤트를 한국어로 요약·해설한다	보안 로그에는 내부 IP 와 계정명이 들어간다. 외부 API 로 보내지 않고 자체 호스팅 하면 데이터가 조직 밖으로 나가지 않는다. 제공자 추상화를 뒤서 필요하면 OpenAI·Anthropic·Gemini 로 바꿀 수 있다

제시 계층 - 사람이 보는 화면

도구	하는 일	왜 이것인가
Spring Boot + PostgreSQL	JWT 인증, 프로젝트 단위 권한, 사용자·공격로그 영속화, ES 조회 API 6종	Elasticsearch 에는 인증·권한 개념이 없다. 누가 어느 프로젝트의 로그를 볼 수 있는가 를 다루려면 관계형 DB 와 애플리케이션 레이어가 필요하다
Next.js 15 React	한국어 대시보드, 로그인·회원가입, 공격 로그 조회, 어택맵 프록시	서버 라우트(/api/*)를 프론트와 같은 프로젝트에 둘 수 있어 ES·Ollama 호출을 브라우저에 노출하지 않고 중계한다
Flask threat-console	순정 T-Pot 설치자용 콘솔 — PDF 리포트, 모델 학습 UI, 모델 업로드	T-Pot 만 쓰는 사용자는 Spring·PostgreSQL 을 원하지 않는다. 의존성 없이 드롭인으로 옅히는 가벼운 콘솔이 따로 필요했다
Docker Compose	10개 컨테이너의 기동 순서·네트워크·볼륨을 한 파일로 정의	<code>docker compose up -d</code> 한 번으로 전체가 재현된다. <code>depends_on: service_healthy</code> 로 의존성 순서를 보장해 백엔드가 DB 보다 먼저 떠서 죽던 문제 를 제거했다

이미 잘 되어 있는 것은 만들지 않는다. 허니팟 운영(T-Pot), 로그 정규화(Logstash), 저장·집계(Elasticsearch)는 검증된 도구에 맡기고, 우리는 **비어 있던 분류·해석·제시 세 층**에만 코드를 썼다. 그 결과 신규 코드는 약 7,800줄이고 T-Pot 본체는 한 줄도 수정하지 않았다.

5-3. ml-classifier – 분류 사이드카

`logstash-*` 를 30초 주기로 폴링해 각 이벤트를 분류하고 `ml-analysis-YYYY.MM.dd` 에 기록한다.

출력 필드	의미
<code>ml_label</code>	Recon / Brute Force / Malware / Intrusion / Etc / Normal
<code>ml_is_attack</code>	라벨 기반 자동 판정 (Etc·Normal 이면 false)
<code>ml_multi_conf</code>	0–100 신뢰도 (규칙 폴백 시 100)
<code>mitre_score</code>	MITRE ATT&CK 위협 점수 0–100
<code>mitre_technique</code>	기법 ID (T1110, T1046 ...)
<code>model_used</code>	<code>rule</code> 또는 <code>ml</code>

특징 추출은 16개 축(시각·포트·프로토콜·허니팟·이벤트유형·로그인시도·명령길이·wget/curl/리버스셸 포함 여부 등)으로 이뤄지며, 학습된 모델이 없으면 `rule_label.py` 의 결정론적 규칙으로 자동 폴백한다. 커서는 `(@timestamp, _seq_no)` 복합 키로 `search_after` 를 쓰므로 같은 밀리초 문서가 누락되거나 무한 재처리되지 않는다.

5-4. llm-analyzer – 해설 사이드카 ● 신규 작성

원래 이 모듈은 별도 브랜치에 존재했으나 **소스가 로컬·백업 어디에도 남아 있지 않았다**. 압축 아카이브 28,246 파일 전수 검색에서도 0건이었다(대조: ml-classifier 93건). 소비자 측 필드 계약에 맞춰 새로 구현했다.

동작은 단순하다. `ml-analysis-*` 중 `mitre_score ≥ 70` 인 문서만 골라 한국어 프롬프트로 LLM 에 보내고, 응답을 `llm-analysis-*` 에 적재한다. 설계에서 신경 쓴 지점은 셋이다.

- **비용 게이트** — LightGBM 추론이 마이크로초라면 LLM 호출은 초 단위다. 점수 게이트로 대상을 좁히고 배치를 작게(기본 20건) 잡는다.
- **중복 과금 차단** — 배치를 분석하기 전에 대상 인덱스에서 기존 `_id` 를 조회해 거른다. `create op-type` 만으로는 중복 쓰기는 막지만 LLM 호출은 이미 낭비된 뒤다.

- **제공자 추상화** — Ollama / OpenAI / Anthropic / Gemini 를 지원하며, 프론트엔드의 `llm-providers.ts` 와 **동일한 환경변수명**을 쓴다. `.env` 하나로 사이드카와 UI 가 같은 백엔드를 본다.

5-5. 제시 계층

Spring 백엔드는 JWT 인증과 프로젝트 단위 권한을 담당하고, ES 조회 6개 엔드포인트를 제공한다. Next.js 프론트는 이를 소비해 한국어 대시보드를 그리며, 어택맵은 `server.js` 의 커스텀 프록시로 `/tpot-map/*` 와 `/websocket` 을 `map_web` 으로 중계한다.

06 핵심 기술 과제 1 – ML 분류기

이 프로젝트에서 가장 많은 시간을 쓴 문제이자, 가장 분명한 결론에 도달한 문제다. 결론은 "현재 데이터로는 ML 이 규칙을 이길 수 없다"이고, 그것을 세 번의 실험으로 측정했다.

6-1. 증상

재학습한 LightGBM 모델(holdout 정확도 0.998)을 배치하자 주입 샘플 60건이 **전부 Recon** 으로 분류됐다. 신뢰도는 96~99%. `mitre_score` 가 32에 머물러 고위험이 0건이 되고, 결과적으로 **LLM 해설 단계가 아무 일도 하지 못했다.**

6-2. 1차 진단 – 라벨이 허니팟의 함수

학습 데이터를 열어보니 라벨이 사실상 `source_honeypot` 하나로 결정되고 있었다.

허니팟	RECON	BRUTE FORCE	MALWARE	INTRUSION	합계
cowrie	49,412	582	6	0	50,000
fatt	50,000	0	0	0	50,000
p0f	50,000	0	0	0	50,000
dionaea	0	0	23,725	0	23,725
sentrypeer	0	18,694	0	0	18,694

18개 허니팟 중 16개가 라벨 하나에 100% 고정이었다. `rule_label.py` 가 대부분의 허니팟에 대해 이벤트 내용과 무관하게 상수를 반환하기 때문이다(`if src_hp == "p0f": return "Recon"`). 게다가 Brute Force 클래스 20,571행 중 **18,694행(90.9%)이 sentrypeer 한 곳에서 나왔다.**

핵심

모델은 편향된 것이 아니라 **존재하는 유일한 신호를 정확히 학습**했다. 라벨이 규칙의 출력이므로, 모델이 도달할 수 있는 최선은 규칙의 재현이다. 기존 기록의 "0.998은 과대평가"라는 메모가 정확했다.

6-3. 2차 시도 – 재균형 (실패)

`rebalance.py` 를 만들어 (라벨 × 허니팟) 셀 단위 상한을 걸었다. 지름길은 실제로 차단됐다.

- Brute Force 내 sentrypeer 비중 **90.9% → 51.6%**
- cowrie 내 Recon : Brute Force **85:1 → 1:1**
- 재학습 holdout 정확도 0.9971

그런데 정규 케이스 검증은 **3/6 으로 재균형 전과 동일**했다. 원인이 라벨 불균형이 아니라는 뜻이었다.

6-4. 3차 진단 – 특징 결손과 train/serve skew

특징 값의 분포를 세어보니 **16개 축 중 6개가 학습 내내 상수 0**이었다.

특징	0 인 비율	원인
login_attempts	100.0%	CSV 에 컬럼 자체가 없음
login_success	100.0%	//
has_reverse_shell	100.0%	비0 0건
has_curl	100.0%	비0 1건
has_wget	100.0%	비0 5건
special_char_cnt	100.0%	비0 53건

공격 유형을 실제로 구분하는 바로 그 축들이 비어 있었다. 원인은 CSV 내보내기 구조였다.

COWRIE.CSV 이벤트	행수	DST_PORT	행동 정보
session.connect	41,725	23.0 / 22.0 ✓	없음
session.closed	41,724	—	없음
login.failed	2,987	전부 빈값	있음
command.input	125	전부 빈값	있음

포트·프로토콜은 session.connect 행에만 있고 행동은 login.failed / command.input 행에만 있다. **둘이 다른 행에 흩어져 있어** 어느 쪽도 완전한 특징 벡터를 만들지 못한다. 결과적으로 같은 이벤트가 학습과 추론에서 이렇게 갈렸다.

	DST_PORT	PROTOCOL	LOGIN_ATTEMPTS
학습 행 (cowrie.login.failed)	0	0	0
실제 ES 문서 (같은 이벤트)	22	6 (ssh)	25

모델은 **실서비스 문서처럼 생긴 입력을 한 번도 본 적이 없었다.**

6-5. 4차 시도 – 세션 조인 (개선, 그러나 천장)

session_join.py 로 라이브 Logstash 가 하는 일을 CSV 에서 재현했다. 세션 connect 행에서 포트·프로토콜을 거둬 같은 세션의 모든 이벤트 행에 전파하고, 세션당 login.failed 수를 세어 login_attempts 를 만들고, 89.9%를 차지하던 세션 복키퍼 행을 (속성을 거둔 뒤) 버렸다.

COWRIE	조인 전	조인 후
행 수	50,000	3,217
dst_port 비0	48.3%	100%
login_attempts 비0	0%	93.3%
라벨 분포	Recon 49,412 / BF 582	BF 3,073 / Recon 128

노이즈가 겹치자 cowrie 의 실제 성격(로그인 실패)이 드러났다. 실제 ES 문서 127건으로 규칙 대비 일치율을 재니:

모델	일치율	주요 오류
v2 (조인 전)	35.4%	BruteForce→Recon 33 · Malware→Recon 24
v3 (세션 조인)	61.4%	Intrusion→Malware 25 · Malware→Recon 24
v4 (+payload 필드)	61.4%	동일

천장의 정체

v4 가 v3 를 넘지 못한 이유는 명확하다. payload 필드에서 리버스셸 패턴 384행을 새로 얻었는데, 그중 **322행의 라벨이 Recon** 이었다. Suricata 의 규칙 라벨은 `alert.category` 에서 오고 그것이 payload 내용과 어긋난다. **특징이 생겨도 라벨이 그 특징을 부정하므로** 모델은 "리버스셸 → Recon" 을 학습한다.

라벨이 규칙에서 나오는 한 모델의 천장은 규칙 재현(100%)이고, 현재 61.4%다. 커면 운영 품질이 나빠진다. **그래서 규칙 모드를 유지한다.**

다만 규칙이 구조적으로 못 하는 것 하나는 모델이 해냈다 — 정상 HTTP/HTTPS 트래픽을 `Normal` 로 100% 신뢰도로 분류한다(규칙은 `Recon` 이라 답한다). 허니팟 특성상 정상 트래픽이 드물어 실익은 제한적이지만, ML 이 더할 수 있는 값이 무엇인지는 분명해졌다.

07 핵심 기술 과제 2 – LLM 판정

LLM 이 한국어 요약은 잘 쓰는데 위험도 판정은 전혀 못 한다는 것을 측정으로 확인하고, 그 일을 모델에서 회수했다.

7-1. 증상

초기 구현에서 `mitre_score` 92(멀웨어)인 문서에 모델이 `severity: LOW` , `risk_score: 3.0` 을 반환했다. 파이프라인이 이미 `≥ 70` 만 통과시키므로 이것은 **내부 모순**이다. 보안 대시보드에서 고위험을 LOW 로 표시하는 것은 적극적으로 위험하다.

7-2. 측정 – 프롬프트 문제인가 능력 문제인가

고치기 전에 원인을 갈랐다. 두 가지는 해법이 다르기 때문이다. `eval_severity.py` 로 고위험 문서 10건에 프롬프트 두 변형을 돌렸다.

프롬프트	하한 미달	가드 불필요	한자 혼입	소요
current	10 / 10	0%	1 / 10	592s
fewshot (판정 예시 3개 추가)	10 / 10	0%	0 / 10	341s

판정 예시를 넣어도 10/10 그대로였다. 프롬프트 문제가 아니라 3B 모델의 능력 한계다.

7-3. 설계 변경 – 잘하는 일만 맡긴다

severity 를 아예 묻지 않고 `mitre_score` 에서 결정론적으로 유도하며, 모델은 잘하는 것(한국어 요약·대응 방안·MITRE 기법 추정)만 담당하게 했다.

- 프롬프트가 짧아지고 실패 모드 하나가 사라진다
- `LLM_SEVERITY=model` 로 더 강한 모델에 판정을 되돌려줄 수 있다. 그때도 하한 가드는 유지된다 — **모델은 올릴 수는 있어도 내릴 수 없다**
- 항상 `true` 라 무의미하던 `severity_source` 를 `severity_source ( score | llm | raised )` 로 교체해, 누가 판정했는지 인덱스에서 바로 읽힌다

설계 원칙으로

이 경험은 일반화된다. LLM 에게 결정론적으로 계산 가능한 값을 묻지 않는다. 요약·설명·번역처럼 언어 능력이 필요한 일만 맡기고, 점수·등급·분류처럼 규칙으로 환원되는 값은 코드가 계산한다. 그래야 실패해도 조용히 틀리지 않는다.

08 조용한 실패 네 건

이번 개발에서 찾아 고친 결함 네 건은 공통점이 있다. 예외도 경고도 없이 잘못된 값이나 빈 화면을 냈다는 것이다. 테스트가 아니라 실측이 이것들을 드러냈다.

8-1. `_intval("23.0")` 이 0을 반환

CSV 는 숫자 컬럼을 `"23.0"` 으로 내보내는데 `int("23.0")` 은 예외를 던지고, 예외 처리가 0을 반환했다. CSV 학습 경로의 모든 포트와 카운트가 0이 되는데 라이브 ES 문서(정수)는 정상이라, 같은 이벤트를 두 경로가 다르게 봤다. §5-4 의 train/serve skew 를 만든 직접 원인 중 하나다.

```
# 수정 전 - "23.0" → ValueError → 0
return int(v or 0)
```

```
# 수정 후 - float 를 먼저 거친다
return int(float(v or 0))
```

8-2. `payload_printable` 미조회

명령 텍스트 추출이 `input` / `command` / `message` 만 읽었는데, Suricata 는 공격 명령을 `payload_printable` 에 담는다. cowrie 가 wget 10행·리버스셸 0행을 낼 때 **Suricata payload** 에는 **891건·443건**이 있었다. 명령 특징의 유일한 실질적 공급원을 놓치고 있었던 셈이다.

8-3. 인덱스 템플릿의 사각지대

소비자들은 `severity` 를 서브필드 없이 집계한다(`terms(field="severity")`). ES 동적 매핑에 맡기면 `text` 가 되어 이 집계가 `Fielddata is disabled` 로 실패하는데, **threat-console** 과 **Spring** 모두 이 **예외를 catch** 해 빈 차트로 삼킨다. 기동 시 인덱스 템플릿을 등록해 `keyword` 로 고정하는 것으로 해결했다.

그런데 같은 함정이 한 번 더 나왔다. **템플릿은 인덱스 생성 시점에만 적용된다**. 나중에 `severity_source` 필드를 추가하자, 이미 만들어져 있던 인덱스에 동적 매핑으로 `text` 가 붙어 또 조용히 실패했다. 기동 시 기존 인덱스에도 템플릿 필드를 push 하고, 타입이 충돌하면 숨기지 않고 경고하도록 고쳤다.

8-4. 한국어에 한자 혼입

Qwen 계열 모델이 한국어 문장 중간에 한자를 섞는다(`악의대시`). 프롬프트에 "한국어로만" 규칙을 넣어도 10건 중 1건에서 발생했다. **출력 검사**로 전환해, CJK 표의문자가 발견되면 파싱 오류로 처리하고 기존 재시도 경로가 다시 시도하도록 했다. 재시도가 소진되면 결정론적 폴백 문구(순수 한국어)로 내려간다. 검증에서 신규 문서 12건 중 한자 혼입 0건.

8-5. ☒ 같은 실패가 다시 나지 않게 – `ml-classifier` 인덱스 템플릿

8-3 은 `llm-analysis-*` 이야기였다. 그런데 **같은 구멍이 `ml-analysis-*` 에도 그대로 남아 있었다** – 분류기에는 인덱스 템플릿이 아예 없어 모든 필드가 동적 매핑이었다. 지금까지 문제가 드러나지 않은 것은 소비자가 우연히 `model_used.keyword` 처럼 하위필드를 쓰고 있었기 때문이지, 설계가 안전해서가 아니었다.

`ml-analysis-*` 는 **하루에 하나씩 새 인덱스**가 만들어진다. 템플릿이 없으면 각 인덱스가 "그날 처음 들어온 문서" 모양대로 타입을 잡으므로, 소비자가 `ml-analysis-*` 로 한꺼번에 조회할 때 **하루는 되고 하루는 안 되는** 상태가 될 수 있다.

타입을 "더 좋은 것"으로 올리지 않은 이유

`src_ip` 를 `ip` 로, `mitre_score` 를 `integer` 로 올리고 싶은 유혹이 있다. 그러면 **오늘 인덱스는 `text` , 내일 인덱스는 `ip` 가 되어** 오히려 날짜별로 갈린다. 템플릿은 **지금 인덱스에 있는 모양 그대로** 고정하는 데 썼다. 타입을 올리려면 전체 `reindex` 가 함께 가야 한다.

실측으로 확인한 효과 — `dest_port` 에 문자열 `"23.0"` 을 넣으면 ES 가 숫자 23 으로 강제변환해 색인하고 (`_source` 에는 원문이 남는다), `"abc"` 는 `document_parsing_exception` 으로 **거부하고 로그에 남긴다**. 어느 쪽이든 필드가 `text` 로 굳어 집계가 죽는 일은 없다. 6-1 의 `int("23.0")` 버그가 저장 계층에서 한 번 더 막힌 셈이다.

기존 인덱스에도 `_ensure_field_mappings()` 로 템플릿 필드를 밀어 넣었고, 충돌 없이 적용됐다 — "있는 모양 그대로" 설계가 맞았다는 확인이다.

09 2차 개발 – 사용자 인터페이스

v1.0 이후 화면을 전면 손봤다. 계기는 두 가지였다. 하나는 화면이 사실과 다른 값을 보여주고 있었던 것이고, 다른 하나는 인터페이스가 프로젝트가 아니라 템플릿을 닮아 있던 것이다.

9-1. 화면이 거짓말하던 두 곳

증상	원인	조치
한 화면에 모델명이 둘	랜딩·대시보드 네 곳이 <code>Powered by Llama 3.1</code> 로 고정돼 있었다. 실제 구동 모델은 달랐고, 헤더 옆 상태 칩은 <code>/api/health</code> 에서 진짜 값을 받아 표시하고 있었다	하드코딩 제거, 서술을 모델 무관하게 변경
모델 카드가 고장난 것처럼 보임	규칙 모드에는 학습 지표가 없어 정확도·F1·CV 가 모두 <code>-</code> , 학습 샘플 <code>0</code> 으로 나열됐다. 지표를 못 불러온 상태와 구분되지 않았다	규칙 모드 분기를 넣어 배지를 "규칙 기반"으로 바꾸고 이유를 문장으로 표시

9-2. 디자인 – 생성형 도구의 흔적 제거

랜딩 페이지는 파일 첫 줄이 `// 랜딩 페이지 – Clean Light SaaS` 였다. 템플릿을 그대로 쓴 결과 다음이 남아 있었다.

- 인디고·보라 강조색(`#4f46e5`)과 그라데이션 제목·CTA 밴드
- 파스텔 아이콘 타일 6칸 그리드, 전부 가운데 정렬
- "무료 시작 / 무료 회원가입 / 지금 바로 무료로 시작하세요" — 판매 문구. **졸업작품에 맞지 않는 어조다**

토큰 한 곳(`globals.css`)을 바꿔 전체에 적용했다.

항목	V1.0	V2.0	이유
강조 색	#4f46e5 인디고	#0f6b78 스틸 시안	계측기 계열. 보고서·폼보드와 동일 아이덴티티
모서리	14 · 10 · 18px	5 · 3 · 7px	둥근 카드가 템플릿 인상의 핵심
서체	시스템 기본	IBM Plex Sans KR · Mono	엔지니어링 문서용으로 설계된 서체
히어로	가운데 정렬 · 그라데이션	좌측 정렬 2단	수치를 나란히 놓아 읽는 순서를 만든다

내용도 다시 썼다. 기능 나열 대신 **문제 → 4단 구조 → 측정 결과 → 스택** 서사로 바꾸고, 수치는 라이브가 아니라 검증 시점 기준값으로 고정했다. 소개 면이 스택 가동 여부에 따라 0 으로 보이면 안 되기 때문이다.

9-3. 대시보드 정보 설계

대시보드는 읽는 문서가 아니라 **훑고 조작하는 화면**이다. 그 기준으로 다시 배치했다.

문제	조치
요약 → 모델카드 → 어택맵(650px) → 작업영역 순서라 정작 일하는 화면이 스크롤 아래에 있었다	요약 → 작업영역 → 모델 → 맵 으로 재배치
지도는 실시간 이벤트가 있을 때만 그려지는데 기본이 펼침이라 대개 빈 화면이 650px 를 차지했다	기본 접힘. 안내문도 사실에 맞게 수정
로그 목록이 <code>maxHeight: 340</code> 으로 묶여 100건 중 5건만 보였고, 바깥 패널에도 스크롤이 걸려 중첩 스크롤 이었다	뷰포트에 맞춰 늘리고 스크롤은 목록에만. 폭도 380→460px
위험도가 배지 글자로만 표시돼 목록을 훑어서는 급한 건이 안 보였다	항목 왼쪽에 위험도 색 띠 — 형태로 읽히게
아무것도 선택하지 않으면 오른쪽 700px 가 안내문 세 줄만 띄우고 비었다	"지금 들어오는 공격" 유형별 분포로 대체. 막대 색은 그 유형에서 관측된 최고 위험등급을 따른다

9-4. 어택맵 – 외부 타일 서비스 정책 변경

지도가 "Initializing" 에서 멈춰 보였다. 추적해보니 데이터 경로는 전부 정상이었다 — `map_data` → Redis
아크 발행 확인, 프론트 프록시 경우 websocket 42프레임 수신, 자산 전부 200.

진짜 원인

CARTO 가 basemap 타일에 API 키를 요구하도록 바뀌었고, 키 없이 요청하면 "API KEY REQUIRED" 워터마크가 인쇄된 타일을 HTTP 200 으로 돌려준다. 실패로 보이지 않아 더 나빴다 — 지도는 뜨는데 글자만 덮여 있었다. 서버도메인 세 곳이 모두 같은 크기(10,861B)의 워터마크 이미지였다.

키가 필요 없는 OpenStreetMap 표준 타일로 교체하고 어두운 톤은 CSS 필터로 만들었다. 테마 전환 함수가 옛 CARTO URL 로 레이어를 찾던 것도 함께 고쳤다 — 그대로 두면 전환 시 레이어가 겹친다.

9-5. 파이프라인만 패턴 단위로 가고 화면이 안 따라갔다

12-5 에서 해설을 패턴당 한 번만 하도록 바꾼 직후, 화면을 보니 **똑같은 문단이 424줄** 반복되고 있었다. 문장 자체는 옳다(IP·시각을 빼둔 덕에 틀린 값도 없다). 다만 **파이프라인은 패턴 단위로 갔는데 화면은 여전히 건별**이었다.

백엔드에 `/api/llm-patterns` 를 더했다 — `llm_group_key` terms 집계 위에 대표 해설(top_hits) · 같은 패턴을 두드린 공격자 수 (`src_ip` cardinality) · 최근 관측 시각을 얻는다. 대시보드는 이걸로 카드 하나씩 그린다.

반복을 없앤 것 이상의 효과

접어놓으니 이 프로젝트가 측정으로 찾아낸 사실이 **화면에 그대로 드러난다** — "고위험 N건이 M개 패턴이고 해설 1건이 평균 108.8건을 덮는다". 그전까지 이 발견은 보고서에만 있었고 화면은 오히려 반복으로 지저분했다.

패턴 키가 없는 옛 문서(패턴 재사용 이전에 건별로 해설된 42건)는 버리지 않고 (건별 해설) 버킷으로 모은다. 다만 그걸 "허니팟|라벨|포트" 로 쪼개면 `포트 -` 같은 **가짜 패턴처럼 보이므로** 화면에서 따로 표시한다 — 합계에서 빼면 숫자가 안 맞는다.

9-6. 자산 노출 대조 — 허니팟 데이터를 "내 얘기"로

14-3 이 네 간극 중 하나로 적어둔 것이다. 허니팟이 본 것은 "우리 대역을 노리는 공격" 이지 "우리가 뿔렸다"가 아니다. 그래서 아무리 많은 공격을 보여줘도 남의 얘기로 읽힌다.

연결 고리는 이미 있었다. 파이프라인이 `dest_port` 를 뽑아두므로 **열어둔 포트 목록만 있으면** 맞댈 수 있다. ML 도 LLM 도 쓰지 않는다.

포트	공격	고위험	공격 IP	판정
445 · SMB	107,551	40,487	228	지금 노려지고 있다
80 · HTTP	16,818	266	828	고위험 있음
6379 · Redis	1,268	522	221	비중이 높다(41%)
3306 · MySQL	1,671	248	219	고위험 있음
22 · SSH	13,801	0	578	두드리지만 고위험 없음
3389 · RDP	1,479	0	296	두드리지만 고위험 없음

수집 창 기준이다. **"안 맞고 있다" 도 답**이므로 공격이 0건인 포트도 함께 돌려준다 — 목록에서 빠지면 조회가 안 된 것인지 공격이 없는 것인지 구분되지 않는다. SSH(22)가 13,801건을 받고도 고위험이 0인 것은 규칙이 로그인 실패 자체를 고위험으로 올리지 않기 때문이고, 그 판단이 화면에 그대로 보인다.

포트 목록을 어디 두는가 — 판단이 한 번 뒤집혔다

처음에는 브라우저(`localStorage`)에만 두었다. 서버에 쌓으면 **"어느 조직이 어떤 포트를 열어놨는가"** 라는 목록을 운영자가 갖게 되고, 이 기능이 팔릴 만한 이유가 바로 그 정보의 민감함이기 때문이다.

그런데 **여러 사람이 한 센서를 공유하는 형태(9-8)에서는 그 선택이 성립하지 않는다**. 기기를 바꾸면 목록이 사라지고, 계정이 있는데도 브라우저마다 다시 적어야 한다. 공격 데이터는 공유하되 대조는 개인별이라야 하므로 **계정에 딸린 저장으로 옮겼다**(`asset_profile`).

민감함은 그대로이므로 취급으로 막는다 — 경로에 사용자 id 를 받지 않아 **남의 목록을 조회할 방법이 없고**, 저장 시 내용을 로그에 남기지 않으며, 길이 상한을 둔다. 실측으로 다른 계정은 빈 값을 본다.

9-8. SaaS 형태로 굴릴 수 있는가

메인 서버가 허니팟을 돌리고 이용자는 브라우저만으로 분석을 받는 형태가 가능한지 확인했다. **구조는 이미 그 모양이다** — 프론트는 허니팟을 전혀 모르고 Spring 만 부르며, Spring 이 ES 를 읽는다. 스트리밍도 이미 `text/event-stream` (SSE)이라 브라우저가 토큰 단위로 받는다.

데이터 격리가 없는 것이 이 형태에서는 맞다

`ThreatConsoleService` 는 사용자·프로젝트로 거르지 않고 `m1-analysis-*` 문서에도 소유자 필드가 없다. 보통은 결함이지만, **센서가 운영자 것 하나뿐인 형태에서는 모든 이용자가 같은 공격 피드를 보는 게 정상**이다. 그러면 관건은 격리가 아니라 **개인화**다 — 같은 445 공격 107,551건이 **445 를 열어둔 사람에겐만** 경고가 되어야 한다. 9-6 의 자산 대조가 정확히 그 층이고, 그래서 이번에 브라우저 저장에서 **계정 저장으로 옮겼다**.

막던 것은 둘이었다.

① **추론 엔드포인트가 인증 없이 열려 있었다.** `/api/analyze/*` 가 토큰 없이 200 을 돌려줬다. 본문이 그대로 프롬프트에 들어가므로 입력도 호출자가 정하고, 외부 제공자 키를 쓰는 구성이면 **남의 크레딧을 태우는 프로кси**가 된다. 10장에서 Spring 쪽 `permitAll` 을 달았는데 **Next 라우트는 같은 감사를 받지 않았다.**

② **추론이 직렬화된다.** 실측이다:

상황	소요
단독 1건	60초
동시 2건 — 첫 번째	60초
동시 2건 — 두 번째	82초

ollama 단일 인스턴스에 CPU 추론이라 줄을 선다. 이용자 열 명이 동시에 누르면 마지막 사람은 10분을 기다리는데 **브라우저는 그동안 아무 말도 하지 않는다.** 기다리게 두느니 바로 거절하는 편이 낫다.

두 라우트에 공용 관문(`lib/추론관문.ts`)을 두어 **인증 · 사용자별 쿼터(기본 10분당 5회) · 동시 실행 상한(기본 2)**을 건다. 거절된 요청은 쿼터를 소모하지 않는다 — 혼잡으로 막힌 사람이 할당량까지 잃으면 이중 처벌이다. 슬롯 반납은 정상 종료·오류·**클라이언트 이탈** 모두에서 하고, 두 번 반납해도 카운터가 음수로 흐르지 않게 막았다(그러면 제한이 서서히 무력해진다).

실측	결과
토큰 없음 · 위조 토큰	401
정상 토큰	200
동시 3건 중 세 번째	503
6회차	429

남은 것. 쿼터가 프로세스 메모리에 있어 재기동하면 초기화되고 복제본 간에 공유되지 않는다 — 단일 인스턴스 전제다. 여러 대로 늘리면 Postgres 나 Redis 로 옮겨야 한다. 그리고 읽기 중심 화면(대시보드·패턴·자산·대조·어택맵)은 전부 ES 조회라 이용자 수에 잘 견디지만, **온디맨드 추론만은 GPU 로 올리거나 외부 API 로 돌리거나 유료 등급에만 여는 정책이 필요하다.** 패턴 해설(12-5)은 미리 만들어두므로 이용자 수와 무관하다.

9-9. 온디맨드 추론 정책 – 부를 일을 없앤다

9-8 이 남긴 마지막 과제다. 선택지를 "GPU · 외부 API · 유료 등급" 셋으로 적어뒀는데, 셋 다 **"그래도 추론을 부를 때"** 의 이야기였다. 먼저 물을 것은 **정말 불려야 하는가** 였다.

12-5 에서 llm-analyzer 는 패턴당 한 번만 모델을 부르도록 바뀌었지만, **결과는 사건마다 문서로 남긴다**. 그러니 게이트를 통과한 고위험 사건은 이미 해설을 갖고 있다. 화면이 그걸 먼저 보면 된다.

경로	소요	비고
저장된 해설 조회	42ms	고위험 12건 표본에서 12/12 적중
온디맨드 추론	60초	약 1,400배 차이

화면은 사건을 고르는 즉시 해설 카드를 띄우고, AI 분석 시작 버튼은 해설이 있으면 더 깊게 분석 (AI 호출) 로 바뀐다 – **추론이 기본이 아니라 선택이 된다**.

공유 해설임을 화면이 말해야 한다

이 문장은 같은 패턴의 여러 사건이 함께 쓰는 것이다. 사건별 맞춤 문장으로 읽히면 **없는 정밀도를 있다고 오해**하게 되므로 "이 유형 공통" 꼬리표와 "저장된 결과 · 추론 없음" 을 함께 띄운다. 7장에서 위험도를 회수하고 12-5 에서 IP·시각을 뺀 것과 같은 방향이다 – **모델이 하지 않은 일을 했다고 보이게 두지 않는다**.

그룹 키를 다시 계산하지 않는다. 임시 포트 접기(32768 이상) 같은 규칙을 Java 에 한 번 더 옮겨 적으면 두 구현이 어긋나는 순간 조용히 틀린 해설을 보여준다 – 9-7 에서 **포트 번호로 정확히 그 실수를 이미 한 번** 했다. 대신 source_doc_id (= llm-analysis-* 문서의 _id)로 바로 찾는다.

남은 것. 해설이 아직 없는 사건을 누를 때인데, 그건 9-8 의 관문이 쿼터·동시 실행으로 막는다. GPU 나 외부 API 는 **그 빈도가 실제로 문제가 될 때** 검토하면 된다.

9-7. 만들다 걸린 것 두 가지

임시 포트 그룹의 해설에 구체적 번호가 박혔다. 패턴 프롬프트가 대표 문서의 dest_port 를 그대로 넣어 모델이 "포트 44930 을 통해" 라고 썼고, 그 문장이 같은 그룹 208건에 붙어 **전부 틀린 번호**가 됐다 – 12-5 에서 IP·시각을 뺀 것과 정확히 같은 실수를 포트에서 반복한 것이다. 임시 포트면 "임시 포트(32768 이상, 사건마다 다름)" 로 넘기고 번호를 지어내지 말라고 지시한다. 잘못된 208건은 지워 다시 만들게 했다.

src_ip 집계가 전량 실패했다. "all shards failed" 로 자산 대조가 통째로 0건을 돌려줬다. 원인은 ml-analysis-* 11개 인덱스 전부에서 src_ip 가 text 로 매핑돼 있던 것 – **인덱스 템플릿은 생성 시점에만 적용된다는**, 8-5 에 기록해둔 바로 그 함정이다. src_ip.keyword 로 집계한다.

10 2차 개발 – 백엔드 결함 일곱 건

백엔드를 훑으며 일곱 건을 고쳤다. 그중 셋은 보안, 둘은 데이터 정합성, 둘은 기능 자체가 연결돼 있지 않은 문제였다.

#	결함	결과
1	JWT 서명 키가 소스에 하드코딩 <pre>private final String SECRET = "my-secret- key-...";</pre>	저장소를 볼 수 있는 누구나 임의 사용자의 토큰을 위조 할 수 있었다. 환경변수로 옮기고, 미설정 시 임의 키 생성 + 경고(재기동 시 토큰 무효 – 설정하라는 신호). 32바이트 미만이면 기동 실패
2	실패가 그럴듯한 분석 결과로 저장	외부 분석 호출이 실패하면 <code>riskScore 50 / MEDIUM</code> 을 만들어 돌려줬고 호출부가 그대로 DB 에 저장 했다. 화면에서 진짜 분석과 구분되지 않는다. <code>Optional</code> 반환으로 바꿔 실패 시 결과를 남기지 않는다
3	HTTP 타임아웃 없음	기본 <code>RestTemplate</code> 은 타임아웃이 없다. 상대가 응답하지 않으면 서블릿 스레드가 무한 대기하고 적재 API 전체가 멎는다. connect 2초 / read 5초
4	모든 오류가 403	서비스가 던진 예외가 Spring 의 <code>/error</code> 로 포워딩되는데 그 경로가 인증 대상이라 403 Forbidden 이 나갔다. "이미 사용 중인 이메일입니다" 같은 메시지가 클라이언트에 도달하지 못했다. 전역 예외 처리기 신설 → 400 + 한국어 메시지
5	N+1 쿼리	프로젝트 로그를 전부 가져온 뒤 건별로 분석 결과를 다시 조회했다. 로그 1,000 건이면 쿼리 1,001번. 배치 조회로 2회 고정
6	로그인·회원가입이 백엔드를 호출하지 않음	가장 큰 기능 결함. 백엔드 JWT 인증은 완전히 동작하는데 프론트가 쓰지 않았다. 회원가입은 <code>setTimeout</code> 으로 성공을 흉내 내 계정이 만들어지지 않았고 , 로그인은 하드코딩된 계정 표로 프론트에서 판정했다 (비밀번호가 브라우저에 노출). 서버 라우트로 중계해 실제 API 에 연결
7	대시보드가 토큰을 쓰지 않음	누구나 대시보드를 열 수 있었다. 진입 시 토큰의 존재가 아니라 유효성 을 서버에 확인하고(만료·서명 키 변경 시 값은 남아 있다), 없으면 로그인으로 보낸다

공통점

일곱 건 중 여섯 건이 **예외도 경고도 없이** 잘못 동작했다. 403 으로 뭉개진 오류, 가짜 분석 결과, 위조 가능한 토큰, 가짜 회원가입 – 모두 화면상으로는 정상으로 보였다. 8장의 "조용한 실패"와 같은 성격이다.

11 ML 활성화 – 라벨러 개선과 교차검증

6장에서 ML 을 켜지 못한 이유는 "모델"이 아니라 "라벨"에 있었다. 라벨러를 고치자 규칙 자체가 좋아졌고, 그 위에서 모델을 규칙을 대체하지 않는 방식으로 운영에 붙일 수 있었다.

11-1. 규칙 라벨러의 결함 네 가지

규칙은 학습 정답이자 동시에 **운영 분류**이다. 여기를 고치면 양쪽이 같이 좋아진다.

#	결함	영향
1	Suricata 판정이 payload 를 전혀 보지 않았다	페이로드에 <code>bash -i >& /dev/tcp/...</code> 가 있어도 <code>alert.category</code> 가 <code>attempted-recon</code> 이면 정찰로 분류
2	<code>network-scan</code> 이 침입 목록에도 있었다	침입을 먼저 검사하므로 네트워크 스캔이 전부 침입이 됐고, 정찰 쪽 항목은 죽은 코드였다
3	<code>severity == 2</code> → <code>Malware</code>	멀웨어라는 신호가 어디에도 없는데 그렇게 불렀다
4	경보 없는 Suricata 행을 정찰로 불렀다	Suricata 는 flow·dns·tls 같은 자체 기록도 남기는데 그런 행엔 카테고리도 심각도도 없다. 실측 20만 행 중 146,441행(73%) . 공격 근거가 0 인 행을 "정찰"이라 부르면 그 라벨의 정의가 무너진다

내용에 확실한 근거(리버스셸·원격 실행파일 다운로드)가 있으면 카테고리보다 우선하도록 바꾸고, 경보 없는 행은 `Etc` (무해·불명)로 분리했다.

11-2. 라벨 분포 변화

라벨	개선 전	개선 후	변화
Recon	518,130	406,311	-111,819
Etc	844	147,285	+146,441
Malware	58,698	23,793	-34,905
Intrusion	8,518	8,801	+283
Brute Force	23,062	23,062	0

전체 609,252행 중 **181,759건(29.8%)** 의 라벨이 바뀌었다. 가장 중요한 지표는 이것이다.

핵심 지표

리버스셸 페이로드인데 `Intrusion` 이 아니던 행: **322/384 → 0/384**. 특징과 라벨이 서로를 부정하던 문제가 사라졌다.

재학습 결과 실제 문서 300건 기준 규칙 대비 일치율이 **59.0% → 79.0%** 로 올랐다(세션 조인 v3 → 라벨 개
선 v5).

11-3. ☒ 활성화 전에 고쳐야 했던 것

저신뢰를 "무해"로 바꾸던 코드

기존 분류기는 모델이 있으면 라벨을 통째로 덮어썼고, 신뢰도가 임계값 미만이면 "Normal" (무해)로 바꿨다. **모델이 처음 보는 공격일수록 무해로 표시되어 고위험 파이프라인에서 사라진다** — 보안 도구가 가면 안 되는 방향이다.

11-4. 새 설계 — 규칙이 기준선, 모델은 교차검증

모델은 라벨을 뒤집지 못한다. 대신 신뢰도와 자기 판단을 남겨 검토 대상을 드러내는 역할을 한다.

MODEL_USED	의미
ml+rule	모델·규칙 합의 — 신뢰도 높은 건
rule(dissent)	모델이 다른 답 — 검토 대상(라벨은 규칙을 따름)
rule(low-conf)	모델이 확신하지 못함 — 규칙
rule	모델 없음 또는 실패

11-5. 전량 재분류 — 100건 표본이 아니라 코퍼스 전체로

v2.0 의 수치는 표본이 편향돼 있었다

v2.0 이 보고한 **합의율 85%** 는 모델을 켜 두어 들어온 100건만 본 값이다. 나머지 1,636건은 모델이 없던 시절에 규칙만으로 분류돼 `model_used=rule` 로 남아 있었다. 100건은 3시간짜리 주입의 마지막 조각이라 허니팟 구성이 전체와 다르다 — **표본이 코퍼스를 대표하지 않는다**.

`ml-analysis-*` 는 `logstash-*` 에서 언제든지 다시 만들 수 있는 파생 인덱스다. 인덱스를 지우고 커서를 초기화해 **1,736건 전량을 재분류**했다. LLM 결과(`llm-analysis-*`)는 별도 인덱스라 영향이 없다.

MODEL_USED	V2.0 (100건 표본)	V2.1 (1,736건 전량)
ml+rule	85	1,414
rule(dissent)	15	322
rule(low-conf)	0	0
합의율	85.0%	81.5%

11-6. 불일치 322건은 전부 같은 한 가지 실패다

불일치를 쪼개보니 **322건 모두 Cowrie** 이고, 규칙은 예외 없이 `Intrusion`, 모델은 `Malware` 295건 · `Recon` 27건이었다. 합의한 1,414건 쪽에는 Cowrie 680 · Heralding 386 · ConPot 348 이 들어 있고, **Heralding 과 ConPot 은 불일치가 한 건도 없다.**

6장에서 진단한 그대로다. 학습셋의 리버스셸 384행이 전부 `suricata` 라 **모델은 코우리 침입을 한 번도 본 적이 없다.** 그래서 아는 것 중 가장 가까운 `Malware` 를 고른다. 오류율 18.5% 는 여러 원인이 섞인 잡음이 아니라 **단일 학습 공백 하나**다.

확신도로는 못 거른다

그 322건에서 모델의 확신도는 **평균 98.9%(최소 86.1%)**였다. `CONF_THRESHOLD=0.5` 는 단 한 건도 걸러내지 못한다. 모델은 모르는 것을 모른다고 하지 않고 **아는 것 중 가장 가까운 답을 자신 있게 고른다.** 11-3 에서 "저신뢰를 무해로" 바꾸던 코드를 걷어낸 것이 옳았음을 여기서 다시 확인했다 — 저신뢰가 아니라 **고신뢰 오답**이 문제였다.

v2.0 의 서술 정정. v2.0 은 불일치를 "전부 규칙 Intrusion 대 모델 Recon" 이라고 썼다. 100건 표본에서는 그랬지만 전량에서는 Recon 은 27건뿐이고 **대다수(295건)가 Malware** 다. 방향(cowrie 리버스셸 공백)은 맞았고 구체적 오답 클래스가 틀렸다.

규칙이 기준선이므로 라벨은 322건 모두 옳게 남았다. 모델이 라벨을 뒤집을 수 있었다면 **실제 침입 322건이 전부 다른 유형으로 바뀌었을 것이다.** 교차검증 구조가 흡수한 오류가 정확히 이만큼이다.

대시보드 모델 카드에는 holdout 정확도 대신 **실제 합의율**을 표시한다. holdout 99.94% 는 정답 라벨을 규칙에서 뽑아 부풀려진 값이라, 맥락 없이 띄우면 9-1 에서 고친 것과 같은 종류의 거짓말이 된다.

11-7. 실 ES 문서로 재학습 – 11-6 의 진단이 맞았다

11-6 은 "오류율 18.5% 는 잡음이 아니라 **단일 학습 공백 하나**" 라고 진단했다. 학습셋이 CSV 내보내기에서 만들어져 특징이 결손됐다는 것이 6장의 지적이었고, 해법으로 적어둔 것이 **실 Elasticsearch 문서로 학습**이었다. 그것을 했다.

AWS 에 띄웠던 T-Pot 이 **2026-04-27 ~ 05-06(10일)** 동안 받은 실제 공격 로그를 T-Pot 의 logstash 설정 그대로 색인했다(13-1). 그중 가장 이른 250,000행으로 학습한 모델이 **v5** 다.

가설이 맞았다. CSV 학습본(v4)에서 상수 0 이던 `has_wget` · `has_curl` · `has_reverse_shell` · `login_attempts` 가 ES 문서에는 들어 있다. 학습에 쓰지 않은 실 문서 200,000건으로 규칙 합의율을 재니 **v4 의 Malware 재현율 3.0% 가 v5 에서 100.0%** 가 됐다 — v4 는 Malware 3,450건 중 3,328건을 Intrusion 으로 불렀는데, 두 클래스를 가르는 특징이 학습셋에서 전부 0이었으니 당연한 결과다.

모델을 "컨" 것이 아니다

11-4 의 설계는 그대로다. 모델은 여전히 라벨을 뒤집지 못하고 `ml_label_raw` 에 자기 판단만 남긴다. 바뀐 것은 **그 자리에 어느 모델이 앉아 있는가** 뿐이다. 실제 선택지는 "ML 을 커느냐" 가 아니라 "어느 모델로 교차검증하느냐" 였고, v4 로 두면 검토 큐가 모델 쪽 오답으로 채워진다.

11-8. v5 교체와 전량 재분류 – 합의율 99.9%

2026-09-15 에 `models/` 를 v5 로 교체하고 1,026,903건을 전량 재분류했다. v4 는 `models/superseded-v4-csv-sessionjoin/` 에 사유와 함께 남겼다.

라벨	건수	v4 이견	v5 이견
Etc	566,099	169	1
Recon	392,668	107,257	101
Malware	24,455	33	414
Brute Force	23,061	18,593	72
Intrusion	20,620	704	194
합의율	1,026,903	87.6%	99.9%

`rule(low-conf)` 는 275 → 0 이 됐다.

Malware 만 늘었다(33 → 414). 전체 이견이 162배 줄어드는 동안 이 라벨만 12배 늘었으므로 그대로 실행한다. 잔여 782건을 쪼개보면 방향이 한쪽이다:

규칙 라벨	이건	모델의 판단	수집 지점
Malware	414	Recon 414	Suricata 411 · Cowrie 3
Intrusion	194	Recon 194	Suricata 194
Recon	101	Intrusion 77 · Malware 24	Suricata 101
Brute Force	72	Recon 42 · Etc 30	Heralding 67 · Cowrie 5
Etc	1	Recon 1	Suricata 1

782건 중 **707건이 Suricata** 이고, 지배적 패턴은 규칙이 Malware · Intrusion 이라 한 것을 모델이 **Recon** 으로 한 단계 낮춰 보는 것이다. 11-6 의 Cowrie 공백처럼 원인이 한 곳에 모여 있어 다음 개선 지점이 분명하다.

교차검증의 값어치는 합의율 숫자가 아니라, 이 782건이 **사람이 볼 큐로 떠오른다**는 데 있다. 11-6 때는 그 큐가 322건이었고 전부 모델 오답이었다.

11-9. 92.5% 는 모델 성능이 아니라 측정 버그였다

재분류 직후 라이브 합의율은 99.8% 인데, 하루 전 홀드아웃으로 낸 값은 92.5% 였다. 학습에 쓰지 않은 구간만 떼어 봐도 99.9% 라 7%p 넘는 차이를 설명할 수 없었다.

원인은 평가셋이었다. `build_dataset.py` 는 호출할 때마다 카테고리 인코더를 **자기가 본 문서에서 새로 만든다**. 홀드아웃을 따로 뽑으면서 인코더도 따로 만들어졌고, 학습셋에만 있던 이벤트 하나가 빠지면서 **그 뒤 코드가 전부 1씩 밀렸다**:

```
event_type 47종 중 22종의 코드가 어긋남
"AST_I20100"   학습=1   홀드아웃=없음   ← 홀드아웃 구간에 이 이벤트가 없다
"CONNECTION_LOST" 학습=2   홀드아웃=1       ← 이후 코드가 전부 1씩 밀림
"NEW_CONNECTION" 학습=3   홀드아웃=2
20만 행 중 60,176행이 모델이 배운 적 없는 코드로 입력됐다
```

학습 인코더 기준으로 되돌려 다시 재니 **micro 99.9% · macro 99.2%** 로, 라이브 측정치와 일치했다.

"Recon → Brute Force 13,880건" 이라던 오답 덩어리는 모델의 약점이 아니라 **밀린 코드**였다.

학습과 평가가 인코딩을 공유하지 않으면 그 평가는 모델을 재지 못한다

이 오차는 모델을 나쁘게 보이게 했으므로 넘어가기 쉬웠다 — 보수적인 숫자는 의심을 덜 받는다. 라이브 측정치와 **어긋난다는 사실**이 아니었다면 찾지 못했을 것이다. 같은 함정을 다시 파지 않도록

`build_dataset.py --encoders-in` 을 추가했다. 기존 인코더를 그대로 쓰고, 그 인코더가 새 문서를 얼마나 덮는지(미지 카테고리 몇 종 · 몇 행)를 경고로 찍는다.

12 LLM 모델 선정

7장에서 위험도 판정을 모델에게서 회수했다. 그렇다면 더 큰 모델을 쓰면 되돌릴 수 있는지, 한국어 품질은 얼마나 다른지를 같은 하네스로 측정했다.

12-1. 후보 세 종 비교

고위험 문서 10건, 동일 프롬프트·동일 조건.

모델	위험도 정확	한자 혼입	MITRE 기법	한국어	라이선스
qwen2.5:3b	0 / 10	1 / 10	✗ T1110·T1059	반복적	Apache-2.0
qwen2.5:7b	2 / 10	1 / 10	✗ T1110·T1059	"마비라벨" 등 깨짐	Apache-2.0
exaone3.5:7.8b	10 / 10	0 / 10	✓ T1105	자연스러움	비상업(NC)

12-2. 3B → 7B 가 0% → 20% 에 그친 의미

발견

위험도는 "점수 92 → CRITICAL" 이라는 단순 대응표인데 모델을 키워도 맞지 않는다. 모델은 "허니팟이 니 실제 침해는 아니다"라고 판단하는 것으로 보이며, 그 자체는 합리적 추론이다. 애초에 LLM 에게 맡길 일이 아니었다는 7장의 결론이 모델 크기와 무관하게 재확인된 셈이다.

같은 사건에 대한 실제 생성 결과를 비교하면 차이가 분명하다.

모델	대응 방안 (발체)
qwen2.5:3b	"...해당 IP 가 SSH 포트 22 에 대한 악성 소프트웨어 시도를 시도하는 것을 확인하면, 해당 IP 를 차단하고, 해당 시도를 시도한 시점의..."
qwen2.5:7b	"203.0.113.55 에서 오는 SSH 연결이 감지되어 마비라벨이 적용되었으며..."
exaone3.5:7.8b	"2026년 9월 9일 새벽 6시 37분경, 공격자 IP 203.0.113.55 가 Cowrie SSH/Telnet 허니팟의 포트 22 를 통해 접근하여 악성 행위를 시도한 것으로 확인되었습니다..."

EXAONE 을 채택했다. 운영 검증에서 10건 전부 성공, 폴백 0, 한자 0, MITRE 기법이 원본과 일치했다.

EXAONE 3.5 는 `License Agreement 1.1 - NC` , 즉 **비상업** 라이선스다. 졸업작품·연구 목적에는 문제가 없으나, 상업화한다면 Apache-2.0 인 qwen2.5 계열로 되돌리거나 외부 API 를 써야 하고 **그 경우 한국어 품질이 눈에 띄게 떨어진다**. 14장에서 다시 다룬다.

12-4. 처리량 실측 — 게이트는 최적화가 아니라 성립 조건이다

품질을 위해 7.8B 를 골랐으므로 속도의 대가를 정확히 알아야 한다. 실 코퍼스(13-1)로 `llm-analyzer` 를 돌려 10건 배치를 두 번 재니 **건당 60.3초 · 61.4초** 였다. 그동안 문서에 적혀 있던 27초는 **3B 기준**이었다.

대상	건수	건당 60초 기준 소요
고위험만 (<code>MIN_SCORE=70</code>)	45,075	31.8일
게이트 없이 전량	1,026,903	725일

하루에 소화할 수 있는 양은 1,416건이다. 수집 10일 동안 45,075건의 고위험이 쌓이는 동안 처리할 수 있는 것은 14,160건이므로 **30,915건이 남는다** — 수집 속도를 따라가지 못한다.

여기서 게이트의 성격이 분명해진다. `MIN_SCORE` 는 비용을 아끼는 장치가 아니라 **이 파이프라인이 성립하기 위한 조건**이다. 게이트가 95.6% 를 막아야 나머지가 겨우 처리 가능한 범위에 들어온다. 실시간 운영으로 가려면 GPU 추론이나 외부 API 가 필요하다(15-2).

12-5. 그런데 31.8일은 같은 이야기를 4만 번 쓰고 있던 것이었다

상업화 검토(14-3)에서 "추론 단가가 단가를 결정하는 지점" 이라고 적고 나서, 그 45,075건이 **서로 다른 사건**인지 세어봤다. 아니었다.

그룹 (허니팟 · 라벨 · 포트)	건수	비중
Dionaea · Malware · 445	22,392	49.7%
Suricata · Intrusion · 445	18,095	40.1%
Suricata · Intrusion · 임시포트	1,558	3.5%
Redishoneypot · Intrusion · 6379	522	1.2%
전체	45,075	74개 그룹

고위험 전체가 **74개 패턴**이고 상위 2개가 89.8% 다. 둘 다 포트 445 — SMB 를 노리는 무차별 웜이다. 건별로 해설한다는 것은 **같은 문장을 4만 번 만드는** 일이었다.

해설 대상	LLM 호출	소요 (61초/건)
건별 (기준)	45,075	31.8일
패턴별 (현행)	74	75분
상위 3개 그룹만	3	3분 → 90% 커버
상위 14개 그룹	14	14분 → 99% 커버

그냥 재사용하면 IP 와 시각이 전부 틀린다

건별 프롬프트는 발생 시각과 공격자 IP 를 주입하고, 모델은 그걸 본문에 그대로 박는다 — "2026년 5월 1일 ..., 공격자 IP 203.0.113.99가 ...". 이 문장을 같은 그룹의 22,391건에 나눠 쓰면 **IP 도 시각도 전부 틀린 값이 박힌다**. 화면에는 멀쩡해 보이므로 8장의 조용한 실패 네 건과 같은 종류다. IP 를 그룹 키에 넣어도(2,238그룹) 시각은 여전히 틀린다.

그래서 패턴 프롬프트에서 **IP 와 시각을 아예 뺐다**. 모델은 "이 허니팟·포트 조합에서 공격자가 무엇을 노리는가" 만 쓰고, 사건별 IP·포트·시각은 문서의 구조화 필드에 이미 있으므로 화면이 그쪽을 읽는다. **모델이 보지 못한 값은 지어낼 수도 없다** — 7장에서 위험도를 회수한 것과 같은 방향의 설계다.

12-6. 임시 포트가 그룹을 잘게 쪼갬다

포트를 원값 그대로 키에 쓰면 그룹이 **1,400개**가 되는데, 그중 **1,287개(92%)가 1건짜리**다. 전부 Suricata|Intrusion|33414 , |35532 , |37352 처럼 임시 포트가 만들어낸 껍데기다 — **전체 사건의 2.86% 인데 해설 호출의 92% 를 먹는다**.

리눅스 임시 포트 범위인 **32768** 위를 한 그룹으로 접으면 74개가 된다. 처음에는 1024 로 접었는데 **그건 틀렸다** — Redis(6379)·MySQL(3306)·MongoDB(27017)·MSSQL(1433) 침입이 무작위 포트 스캔과 한 해설을 쓰게 되어 내용이 뭉개졌다. 경계를 32768 로 올려 서비스 포트는 각자 그룹을 유지한다.

12-7. 실측

배치 10건 단위로 캐시가 차는 과정이 그대로 보인다:

배치	LLM 호출	재사용	비고
1차	7	3	캐시가 비어 있어도 같은 배치 안에 중복이 있다
2차	4	6	
3·4차	0	10	배치 소요 55초 = 폴링 주기 그 자체(추론 0)
재기동 직후	1	9	색인된 패턴 해설을 찾아 쓴다

문서마다 `llm_group_key` 와 `llm_group_reused` 를 남겨 절감을 셀 수 있게 했고,
`LLM_GROUP_REUSE=false` 로 건별 해설로 되돌릴 수 있다. 오류·폴백 0.

13 검증 결과

13-1. 실 공격 로그 전수 검증 (2026-09-15)

v2.1 까지의 검증은 **시연용 주입 데이터**로 한 것이었다(13-2). AWS 에 띄웠던 T-Pot 이 **2026-04-27 ~ 05-06(10일)** 동안 실제로 받은 공격 로그를 전부 색인해 파이프라인을 처음부터 끝까지 완주시켰다.

단계	건수	비고
logstash-*	1,026,903	수집 창 한정 · 수집 지점 18종(허니팟 16 + Suricata + Fatt)
ml-analysis-*	1,026,903	전량 분류 · 공격 판정 460,804 · 고위험(≥ 70) 45,075
고유 공격 IP	24,346	geoip 적용 840,260 (81.8%)

출발지 상위는 미국 299,877 · 브라질 99,817 · 영국 55,733 · 중국 47,853 순이다. 수집 지점별로는 Suricata 727,868 · Cowrie 92,859 · Honeytrap 79,092 · 기타 15종 127,084 로, Suricata 한 대가 전체 의 70.9% 를 만든다 — 허니팟이 아니라 네트워크 전체를 보는 센서라서 그렇다. p0f 170만 행은 수동 핑거프 린팅이라 색인에서 뺐다.

집계 창을 고정해야 재현된다

위 수치는 `ml-analysis-*` 를 **수집 창(04-27 ~ 05-06)으로 한정해** 센 값이다. 인덱스 전체를 그냥 세 면 시연 표본과 지금 들어오는 라이브 트래픽이 함께 잡혀 매일 움직인다. 창 바깥으로 `logstash-1970.01.01` 15,376건이 더 있는데, honeytrap 의 `attackers.json` (누적 집계 파일)이라 타임스탬 프가 없어 epoch 0 으로 떨어진 것이고 개별 공격 이벤트가 아니므로 분류 대상이 아니다.

검증 항목	결과
규칙 기반 분류 분포	Etc 566,099 · Recon 392,668 · Malware 24,455 · Brute Force 23,061 · Intrusion 20,620
모델·규칙 교차 검증	<code>ml+rule</code> 1,026,121 · <code>rule(dissent)</code> 782 · <code>rule(low-conf)</code> 0 → 합의율 99.9% (11-8)
백엔드·프론트	<code>/</code> · <code>/login</code> · <code>/dashboard</code> · <code>/tpot-map/</code> 및 백엔드 <code>health</code> · <code>overview</code> · <code>ml-stats</code> · <code>attacks</code> · <code>model</code> 전부 200
인증 차단	토큰 없이 <code>overview</code> · <code>ml-stats</code> · <code>llm-recent</code> · <code>export/ml</code> 전부 403
어택맵 websocket	101 Switching Protocols
재현	<code>./setup.sh</code> 단독 실행으로 클린 상태에서 exit 0 (13-4)

LLM 해설은 전량을 대상으로 하지 않는다. 고위험 45,075건을 모두 해설하려면 건당 60초 기준 31.8일이 걸린다(12-4). 시연에서는 필요한 만큼만 쌓는다.

13-2. 파이프라인 실측 (시연 스택)

v1.0 의 13-1 은 원본 스택(`capstone/` , 222건)을 켜 것이었다. 2차 개발본은 `capstone-dev/` 에서 돌아가므로 개발본을 다시 켜다. 아래는 **시연용 주입 데이터 1,736건** 기준이며, 13-1 의 실 코퍼스와는 별개의 측정이다. 교차검증 수치(합의율 81.5%)는 v4 모델 시절의 값으로, v5 교체 후에는 11-8 을 따른다.

단계	건수	비고
logstash-*	1,736	수집 이벤트
ml-analysis-*	1,736	전량 분류 · 고위험(≥ 70) 669
llm-analysis-*	340	<code>llm_status=ok</code> 340 · 폴백 0 · 한자 혼입 0

검증 항목	결과
규칙 기반 분류 분포	Brute Force 719 · Recon 348 · Malware 347 · Intrusion 322
모델·규칙 교차검증	<code>ml+rule</code> 1,414 · <code>rule(dissent)</code> 322 · <code>rule(low-conf)</code> 0 → 합의율 81.5%
백엔드 엔드포인트	7종 전부 HTTP 200 — <code>health</code> · <code>overview</code> · <code>ml-stats</code> · <code>llm-recent</code> · <code>llm-stats</code> · <code>export/ml</code> · <code>export/llm</code>
프론트 4 페이지	<code>/</code> · <code>/login</code> · <code>/signup</code> · <code>/dashboard</code> 전부 200
어택맵 websocket	101 Switching Protocols, 12프레임 수신
severity 집계	CRITICAL 173 · HIGH 167 — 전부 <code>severity_source=score</code> (7장 설계대로 점수에서 계산)
export CSV	ml 1,737행 · llm 341행 (헤더 포함, HTTP 200)
해설 모델 혼재	<code>qwen2.5:3b</code> 280 · <code>exaone3.5:7.8b</code> 60 — 모델 교체(12장) 이전 문서가 남아 있는 상태다. 재분석하지 않은 것은 이전 결과를 덮어쓸 이유가 없어서 이고, <code>llm_model</code> 필드로 어느 모델이 쓴 해설인지 구분된다

13-3. 어택맵 복구

어택맵은 두 가지 이유로 죽어 있었다. 첫째, 컨테이너 3종이 `Created` 상태로 한 번도 기동된 적이 없었고 소속 네트워크도 삭제돼 있었다. 둘째, 더 중요하게, `map_data` 는 `logstash-*` 를 `exists: geoip.ip` 로 필터한 뒤 좌표 필드를 읽는데 **주입 문서에 그 필드가 전혀 없었다**. 컨테이너를 compose 에 편입하고 주입 스크립트에 지오 보강을 추가해, Redis 채널로 아크 메시지가 흐르고 프론트 프록시 경유 websocket 으로 수신되는 것까지 확인했다.

13-4. 재현 절차

개발본은 `setup.sh` 하나로 준비된다. 환경 점검 → `.env` 생성 (JWT 서명 키 포함) → 이미지 빌드 → 기동 → LLM 모델 내려받기 → 시연 데이터 주입까지 한 번에 하고, 여러 번 실행해도 안전하다.

```
cd T-POT_PR/capstone-dev
./setup.sh # 준비 + 기동 + 시연 데이터
./setup.sh --check # 환경만 점검하고 종료
```

```
# 접속: 대시보드 :8002 · 백엔드 :8091 · ES :19300 · 어택맵 /tpot-map/
```

```
# 시연 계정: demo@jsp.test / demo1234
```

```
# 어택맵 아크는 "실시간으로 들어오는" 이벤트만 그린다(최근 10초 창).
```

```
# 한 번 주입한 데이터로는 아크가 생기지 않으므로 별도 터미널에서:
```

```
python3 ../integration-tests/demo_feed.py --host http://127.0.0.1:19300
```

샘플 주입 스크립트가 항상 현재 시각으로 문서를 넣으므로, 이전에 필요하던 "ES 타임스탬프 재조정" 절차가 사라졌다.

전량 재분류가 필요할 때(11-5 처럼)는 `ml-analysis-*` 인덱스를 지우고 `tpotce/data/ml-classifier/cursor.json` 을 삭제한 뒤 분류기를 재기동한다. `ml-analysis-*` 는 `logstash-*` 에서 다시 만들어지는 파생 인덱스라 원본 손실이 없고, `llm-analysis-*` 는 별도 인덱스라 영향을 받지 않는다.

14 상업화 가능성

"확장 가능하다"보다 "이 조건이면 가능하고 이걸 막힌다"가 정확한 답이다. 라이선스·단가·기능 세 축으로 정리한다.

14-1. 시장은 실재한다

허니팟을 상품화한 사례가 이미 있다(Thinkst Canary 등, 연 수천 달러 규모). 전담 보안 인력이 없는 조직이 "우리 대역이 실제로 공격받고 있는가"를 알고 싶어 한다는 수요는 검증돼 있다. 이 프로젝트의 차별점은 **한국어 해설**이며, 국내 중소기업·대학·연구실에 유효한 각도다.

14-2. 라이선스가 사업 모델을 결정한다

구성요소	라이선스	제약
T-Pot	GPL-3.0	파생물을 배포하면 소스 공개 의무 — 패키지 판매가 사실상 막힌다
Elasticsearch	SSPL / ELv2	ES 자체를 서비스로 재판매하는 것이 제한된다(내부 사용은 무방)
EXAONE 3.5	NC (비상업)	상업적 사용 불가 — 교체 필요
LightGBM · Next.js · Spring	MIT / Apache-2.0	제약 없음

사업 모델	GPL 대응	판단
관제 서비스(MSSP) — 우리가 운영하고 고객은 보고서를 받는다	SaaS 는 "배포"가 아니므로 공개 의무 없음	• 가장 유력
구축·유지보수 과금 — 오픈소스 그대로 제공하고 인건비로 과금	공개 의무를 애초에 지지 않는다	• 진입 쉬움
제품 패키지 판매	소스 공개 의무 발생	• 막힘

14-3. 현재 상태와 상용 사이의 간극

간극	내용
분류 확장성	분류가 규칙 기반이라 새 공격 유형이 나오면 사람이 규칙을 추가 해야 한다. ML 은 아직 교차검증 역할에 머문다(11장)
추론 단가	CPU 기준 건당 27~60초다. 유사 사건을 묶는 것으로 이 항목은 크게 해소됐다 — 고위험 45,075건이 74개 패턴이라 전량 해설이 31.8일에서 75분 이 됐다(12-5). 단가는 이제 트래픽 양이 아니라 패턴 다양성 에 비례한다. 새 패턴 유입이 분당 1개를 넘으면 그때 GPU 가 필요하다
운영 요건	멀티테넌시·SLA·데이터 격리·감사 로그가 없다. 프로젝트 단위 권한 골격만 있다
고객 체감 긴급성	허니팟 데이터는 "우리 대역을 노리는 공격"이지 "우리가 뚫렸다"가 아니다. 9-6 에서 연결했다 — 열어둔 포트를 대면 "445 를 노린 공격 107,551건 중 고위험 40,487건, 당신은 그 포트를 열어두고 있다"가 된다. 파이프라인이 이미 <code>dest_port</code> 를 뽑고 있어 ML 도 LLM 도 필요 없었다. 남은 것은 자산 목록을 사람이 넣지 않아도 되게 하는 일이다

정리하면 패키지 판매는 GPL 때문에 막히고 관제 서비스형이 현실적이다. 네 간극 중 **추론 단가는 12-5 에서, 고객 체감 긴급성은 9-6 에서 해소됐다**. 남은 것은 **분류 확장성과 운영 요건** 둘이다.

코퍼스가 말해주는 것 — 포지셔닝을 바꿔야 한다

고위험의 89.8% 가 포트 445 를 노린 SMB 씬이다(12-5). 이걸 특정 조직을 **표적한 공격이 아니라** 인터넷을 무차별로 굶는 배경 잡음에 가깝다 — 누가 센서를 올려도 비슷하게 나온다. 그러므로 "우리 대역을 노리는 공격을 알려준다" 는 문구는 과장이 된다.

정직한 각도는 **"노출된 자산이 실제로 무엇에 얻어맞고 있고, 그게 당신 환경에 해당되는가"** 다. 파이프라인은 이미 `dest_port` 를 뽑고 있으므로, 고객 자산 목록과 맞대면 "445 를 노린 공격 40,487건 — 당신 서버 3대에 445 가 열려 있다" 가 된다. **간극 넷 중 가장 값이 큰 연결이고 ML 도 LLM 도 필요 없다.**

진입 순서도 다시 보인다. 14-2 는 관제 서비스(MSSP)를 유력하다고 했는데, 남은 간극 중 셋(멀티테넌시·SLA·데이터 격리)이 전부 MSSP 요건이다. 2~4주 센서 설치 → 한국어 리포트 1부 → 종료 형태의 일회성 진단이면 지금 있는 것만으로 가능하고, GPL 도 무관하며(서비스), 운영 요건이 필요 없다. 레퍼런스가 쌓인 뒤 관제로 넘어가는 편이 순서에 맞는다.

15 한계와 개선 방향

15-1. ML 분류기 – 실 데이터 학습

현재 한계

규칙 대비 61.4%. 라벨이 규칙에서 파생되므로 모델의 천장은 규칙 재현이고, CSV 내보내기의 구조적 결손(§5-4)은 세션 조인으로도 완전히 메워지지 않는다.

개선 방향: 허니팟을 실제로 운영해 `logstash-*` 에 데이터를 쌓고 `build_dataset.py --from-es` 로 학습한다. Logstash 가 이미 포트·프로토콜·행동 필드를 한 문서에 합쳐놓으므로 train/serve skew 자체가 발생하지 않는다. 이것이 ML 을 결 수 있는 유일한 남은 경로다.

더 근본적으로는 **규칙과 독립적인 정답 라벨**이 필요하다. 현재 구조에서는 모델이 규칙을 모방하는 것 이상을 할 수 없다. 공개 침해 데이터셋과의 결합이나 수동 라벨링 표본이 대안이 될 수 있다.

15-2. LLM – 모델 상향

3B 모델은 한국어 산문은 쓸 만하지만 위험도 판정은 전혀 못 한다(§6-2). 호스트 RAM 7.8GB 제약으로 8B 모델은 OOM 이 났다.

개선 방향: 메모리 확보 후 8B 재시도 또는 외부 API (`LLM_PROVIDER=anthropic` 등) 전환. 전환 후 `LLM_SEVERITY=model` 로 되돌리면 `severity_source` 분포가 곧바로 효과 측정치가 된다 (`11m` 비율 이 높을수록 모델이 제 몫을 한다는 뜻).

처리 속도가 가장 큰 과제다. CPU 추론이 **건당 약 60초**(EXAONE 3.5 7.8B 실측)라 하루에 1,416건이다. 실 코퍼스의 고위험 45,075건을 전부 해설하려면 **31.8일**이고, 게이트가 없으면 725일이다(12-4). 10일치 수 집분조차 30,915건이 밀린다 – **실제 T-Pot 트래픽을 실시간으로는 따라갈 수 없다**. GPU 추론, 게이트 상향, 동시 처리, 또는 외부 API 가 필요하다.

15-3. 규칙 라벨러 자체의 개선

§5-2 에서 드러났듯 `rule_label.py` 는 대부분의 허니팟에 대해 내용을 보지 않고 상수를 반환한다. 이는 ML 의 천장을 만드는 원인이기도 하다. cowrie 처럼 이벤트 내용을 실제로 분기하는 로직을 다른 허니팟으로 확대하면 규칙 자체의 품질도, 그 규칙에서 파생되는 라벨의 품질도 함께 올라간다.

15-4. 운영 측면

- **실 T-Pot 연동** — 현재 검증은 합성 샘플 기반이다. `.env` 의 `ES_HOST` 만 바꾸면 실 T-Pot ES 로 전환되도록 설계돼 있으나 실제 연동 검증은 남아 있다.
- **리소스** — 단일 호스트 7.8GB 로 ES·Postgres·Ollama·10개 컨테이너를 모두 돌리기에는 빠듯하다. 시연 전 무관 컨테이너 정리가 필요하다.
- **인증·권한** — Spring 의 프로젝트 단위 권한 모델은 구현돼 있으나 대시보드와의 연결은 부분적이다.

16 결론

영문 허니팟 로그를 한국어 위협 보고로 바꾸는 4단 파이프라인을 구축하고 실패데이터로 동작을 확인했다. 다만 그 과정에서 얻은 가장 큰 성과는 파이프라인 자체가 아니라, 무엇이 되고 무엇이 안 되는지를 측정으로 가른 것이다.

16-1. 달성한 것

- T-Pot 본체를 수정하지 않는 **사이드카 4종**과 인덱스 3단 분리 구조
- 유실됐던 **llm-analyzer 모듈 재작성**(939줄) — 이전까지 해설 인덱스에 쓰는 주체가 아예 없었다
- `docker compose up -d` 한 번으로 재현되는 **자립형 데모 스택**
- 규칙 기반 분류 + 한국어 LLM 해설 + 대시보드 + 실시간 공격 지도의 **end-to-end 동작**
- 조용히 실패하던 **결함 4건** 발견·수정

16-2. 측정으로 가른 것

질문	답	근거
ML 분류기를 컬 수 있는가	● 아니오	규칙 대비 61.4%. 라벨이 규칙 파생이라 천장이 규칙 재현
재균형으로 해 결되는가	● 아니오	편중 해소해도 정규 케이스 3/6 동일 — 원인은 특징 결손
세션 조인이 효과가 있는가	● 부분	35.4% → 61.4% (+26pp), 그러나 약라벨 모순으로 정체
LLM 이 위험 도를 판정할 수 있는가	● 아니오	3B 모델 10/10 실패, few-shot 도 10/10 — 능력 한계
LLM 이 한국 어 해설을 쓸 수 있는가	● 예	60건 status=ok, 폴백 0, 한자 혼입 0/12
라벨러를 고치 면 ML 이 나아 지는가	● 예	규칙 대비 59.0% → 79.0%. 리버스셀 라벨 모순 322/384 → 0/384
그래서 ML 이 규칙을 대체할 수 있는가	● 아니오	합의율 99.9% (실 공격 로그 1,026,903건 전량, v5 모델). 불일치 782건 중 707건이 Suricata 이고 방향이 한쪽이다. v4 모델 시절에는 87.6%, 시연 표본 1,736건에서는 81.5%(불일치 322건 전부 Cowrie · 모델 확신도 평균 98.9%)였다
더 큰 모델이 위험도 판정을 고치는가	● 아니오	3B 0/10 → 7B 2/10. 대응표를 모델에게 맡긴 것이 애초의 오류였다
한국어 특화 모델은 다른가	● 예	EXAONE 3.5: 위험도 10/10, 한자 0/10, MITRE 기법 정답
상업화가 가능 한가	● 조건부	GPL-3.0 때문에 패키지 판매는 막힌다. 관제 서비스형은 가능
콘솔 둘을 통 합해야 하는가	● 아니오	배포 대상이 다름. 겹치는 7개는 읽기 전용이며 드리프트 없음

16-3. 배운 것

세 가지가 반복해서 나타났다.

- **높은 정확도가 동작을 뜻하지 않는다.** holdout 0.998 인 모델이 실제 문서 6건 중 3건을 틀렸다. 학습 분포 안의 수치와 실서비스 입력에 대한 성능은 다른 이야기다.
- **조용한 실패가 가장 비싸다.** 네 건 모두 예외 없이 잘못된 값을 냈고, 그중 둘은 예외를 *잡아서* 빈 화면으로 바꾸는 코드 때문에 더 오래 숨었다.
- **같은 함정이 반복된다.** 인덱스 템플릿을 도입해 막았던 "조용히 빈 차트"가 필드를 하나 추가하자 그대로 재발했다(템플릿은 인덱스 생성 시점에만 적용된다). 한 번 고친 문제는 *구조적으로* 못 생기게 만들어야 한다.
- **고치기 전에 재는 것이 빠르다.** LLM severity 는 프롬프트를 고칠 뻔했으나, 10건을 재보니 프롬프트 문제가 아니었다. 측정이 아니었다면 효과 없는 수정에 시간을 썼을 것이다.

정사평 2026-09-15 · main 컨테이너 10 · 파이프라인 4단 · 실 공격 로그 1,026,903건

부록 A 판별 이력

이 보고서는 판을 나눠 관리한다. 개선이 있을 때마다 기존 판을 덮어쓰지 않고 새 판을 만들어, 어느 시점에 무엇이 사실이었는지 되짚을 수 있게 한다.

판	일자	장	주요 변경
v1.0	2026-09-09	11	최초 작성. 허니팟 개념·목표, 아키텍처, 기술 스택, ML·LLM 두 기술 과제, 조용한 실패 네 건, 검증 결과, 개선 방향
v2.0	2026-09-09	16	2차 개발분 반영. A-1 참조
v2.1	2026-09-09	16	전량 재분류로 교차검증 재측정, 인덱스 템플릿 추가. A-2 참조
v2.2	2026-09-15	16	검증을 실 공격 로그 1,026,903건으로 확대, 교차검증 모델 v4 → v5 교체, 평가 방법 버그 정정, LLM 처리량 실측. A-3 참조
v2.3	2026-09-15	16	패턴 단위 해설로 LLM 호출 99.84% 절감(31.8일 → 75분), 상업화 포지셔닝 재검토. A-4 참조
v2.4	2026-09-16	16	화면을 패턴 단위로(9-5), 자산 노출 대조로 "고객 체감 긴급성" 간극 해소(9-6). A-5 참조
v2.5	2026-09-16	16	SaaS 형태 검토와 무인증 추론 엔드포인트 차단, 자산 목록을 계정 단위로(9-8). A-6 참조
v2.6	2026-09-16	16	온디맨드 추론 정책 — 저장된 해설을 먼저 써 추론을 기본에서 뺐다(9-9). A-7 참조

A-1. v1.0 → v2.0 변경 내역

구분	내용
신규 장	09 사용자 인터페이스 · 10 백엔드 결함 일곱 건 · 11 ML 활성화 · 12 LLM 모델 선정 · 14 상업화 가능성
사실 정정	해설 모델 qwen2.5:3b → EXAONE 3.5 7.8B · 필드 severity_adjusted → severity_source · ML 상태 "비활성" → "규칙과 교차검증"
수치 갱신	규칙 대비 일치율 61.4% → 79.0% · 모델·규칙 합의율 85% 신규 · 라벨 모순 322/384 → 0/384
유지	1~8장의 진단과 결론은 그대로다. v2.0의 작업은 그 진단을 근거로 진행됐고, 결과가 진단을 반박하지 않았다

A-2. v2.0 → v2.1 변경 내역

구분	내용
신규 절	8-5 <code>ml-classifier</code> 인덱스 템플릿 · 11-5 전량 재분류 · 11-6 불일치 322건 분석
사실 정정	불일치의 성격 — v2.0 "전부 규칙 Intrusion 대 모델 <code>Recon</code> "은 100건 표본에서만 참이었다. 전량에서는 <code>Malware</code> 295 · <code>Recon</code> 27 로 대다수가 Malware 다. 진단 방향(cowrie 리버스셸 학습 공백)은 맞았다
수치 갱신	모델·규칙 합의율 85%(100건 표본) → 81.5%(1,736건 전량) · 불일치 15 → 322 · 모델 관여 100건 → 1,736건
새 측 정	불일치 322건의 허니팟 분포(전부 Cowrie) · 불일치 건의 모델 확신도 평균 98.9%(최소 86.1%) · Heralding·ConPot 불일치 0건
유지	1~10장, 12~16장은 그대로다. 합의율이 내려간 것은 구조가 나빠져서가 아니라 표본이 아니라 전량을 켜기 때문 이다

A-3. v2.1 → v2.2 변경 내역

구분	내용
신규 절	11-7 실 ES 문서 재학습 · 11-8 v5 교체와 전량 재분류 · 11-9 평가 방법 버그 정정 · 12-4 LLM 처리량 실측 · 13-1 실 공격 로그 전수 검증
검증 규모	시연 주입 데이터 1,736건 → 실 공격 로그 1,026,903건 (AWS T-Pot 이 2026-04-27~05-06 10일간 수집). 기존 시연 스택 측정은 13-2 로 남겨 두 측정을 구분한다
수치 갱신	합의율 81.5%(1,736건) → 87.6%(v4·실 코퍼스) → 99.9%(v5·실 코퍼스) · 불일치 322 → 782 · LLM 건당 27초(3B) → 약 60초 (7.8B)
사실 정정	홀드아웃 92.5% 는 모델 성능이 아니라 측정 버그 였다. 평가셋을 학습과 다른 인코더로 만들어 <code>event_type</code> 47종 중 22종의 코드가 밀렸고, 20만 행 중 60,176행이 모델이 배운 적 없는 코드로 입력됐다. 교정 후 micro 99.9% · macro 99.2% 로 라이브 측정치와 일치했다(11-9)
새 측정	잔여 이견 782건의 방향 분해(707건이 Suricata, Malware-Intrusion → Recon) · 고유 공격 IP 24,346 · geoip 적용 81.8% · 수집 지점별 분포 · 게이트 없을 때 725일
유지	1~10장, 14~16장의 구조와 설계 판단은 그대로다. 11-4 의 "규칙이 기준선, 모델은 교차검증" 설계도 바뀌지 않았다 — 그 자리에 앉는 모델만 바뀌었다
v2.1 진단의 결과	v2.1 이 "18.5% 오류는 잡음이 아니라 단일 학습 공백 하나" 라고 한 진단은 맞았다 . 실 문서로 학습하니 그 공백이 사라졌다(11-7)

A-4. v2.2 → v2.3 변경 내역

구분	내용
신규 절	12-5 패턴 단위 해설 · 12-6 임시 포트가 그룹을 쪼갬다 · 12-7 실측
수치 갱신	고위험 전량 해설 31.8일 → 75분 (LLM 호출 45,075 → 74) · 상위 3개 패턴이면 3분에 90% 커버
사실 정정	v2.2 의 "31.8일" 은 틀리지 않았지만 질문이 틀렸다 . 45,075건이 서로 다른 사건인지 세어보지 않고 건당 비용만 곱했다. 실제로는 74개 패턴이고 상위 2개가 89.8% 다
새 측 정	패턴 분포(74개) · 커버리지 곡선(3개→90%, 14개→99%) · 임시 포트가 만든 싱글턴 1,287개(전체의 2.86% 인데 호출의 92%) · 배치별 재사용률(7/3 → 4/6 → 0/10)
설계 판단	패턴 프롬프트에서 IP·시각을 제거했다. 그대로 재사용하면 22,391건에 틀린 IP 와 시각이 박히는데, 화면에는 멀쩡해 보인다 — 8장의 조용한 실패와 같은 종류다. 모델이 못 본 값은 지어낼 수 없다는 점에서 7장(위험도 회수)과 같은 방향이다
상업화 재검토	14-3 의 네 간극 중 추론 단가는 해소됐다. 고위험의 89.8% 가 SMB 웹 (표적 공격이 아닌 배경 잡음)이라는 사실에서 포지셔닝을 "노출된 자산이 무엇에 얻어맞는가"로 좁혔고 , 진입도 MSSP 가 아니라 일회성 진단이 먼저라고 판단했다
유지	게이트가 성립 조건이라는 12-4 의 결론은 그대로다 — 게이트 없이 1,026,903건은 여전히 불가능하다. 1~11장·13·15·16장도 그대로다

A-5. v2.3 → v2.4 변경 내역

구분	내용
신규 절	9-5 화면을 패턴 단위로 · 9-6 자산 노출 대조 · 9-7 만들다 걸린 것
간극 해소	14-3 의 네 간극 중 고객 체감 긴급성 을 9-6 이 메웠다. 추론 단가(v2.3)와 합쳐 둘이 해소됐고, 남은 것은 분류 확장성과 운영 요건이다
새 측정	포트별 노출(445 공격 107,551·고위험 40,487·IP 228 / 6379 는 공격 1,268 중 고위험 522 로 비중 41% / 22·3389 는 두드리지만 고위험 0)
설계 판단	자산 포트 목록을 서버에 두지 않는다. 쌓으면 "어느 조직이 어떤 포트를 열어놨는가" 라는 목록을 운영자가 갖게 된다 — 이 기능이 팔릴 이유가 바로 그 정보의 민감함이므로 보관하지 않는 편이 맞다
같은 실수 반복	임시 포트 그룹 해설에 대표 문서의 포트 번호가 박혀 208건이 전부 틀린 번호를 달았다. 12-5 에서 IP·시각을 뺀 것과 같은 실수를 포트에서 되풀이 한 것이다(9-7)
유지	12장의 패턴 단위 해설 설계와 13장 실측은 그대로다. 1~8·10~16장도 변동 없다

A-6. v2.4 → v2.5 변경 내역

구분	내용
신규 절	9-8 SaaS 형태로 굴릴 수 있는가
보안	<code>/api/analyze/*</code> 가 인증 없이 200 을 돌려줬다. 10장에서 Spring 쪽 <code>permitAll</code> 을 달았으나 Next 라우트는 같은 감사를 받지 않았다 . 인증·쿼터·동시 실행 관문을 걸었다
판단 반복	9-6 은 "포트 목록을 서버에 두지 않는다" 고 적었다. 여러 사람이 한 센서를 공유하는 형태에서는 그 선택이 성립하지 않아 계정 저장으로 옮겼다 . 민감함은 취급으로 막는다 — 경로에 사용자 id 없음 · 내용 미로깅 · 길이 상한. 무엇이 왜 바뀌었는지 9-6 에 남겼다
새 측정	추론 직렬화(단독 60초 / 동시 2건의 두 번째 82초) · 관문 응답(401 · 503 · 429)
유지	데이터 격리가 없는 것은 이 형태에서 결함이 아니다 — 센서가 하나이므로 모든 이용자가 같은 피드를 본다. 관건은 격리가 아니라 개인화이고 그것이 9-6 이다

A-7. v2.5 → v2.6 변경 내역

구분	내용
신규 절	9-9 온디맨드 추론 정책
질문이 틀렸던 것	9-8 은 선택지를 "GPU · 외부 API · 유료 등급" 셋으로 적었다. 셋 다 "그래도 추론을 부를 때" 의 이야기이고, 먼저 물을 것은 정말 불러야 하는가 였다. 12-5 가 사건마다 문서를 남기므로 고위험 사건은 이미 해설을 갖고 있다
새 측정	저장된 해설 조회 42ms · 고위험 12건 표본 12/12 적중 · 온디맨드 추론 60초(약 1,400배)
설계 판 단	공유 해설임을 화면이 말한다("이 유형 공통" · "저장된 결과 · 추론 없음"). 사건별 맞춤 문장으로 읽히면 없는 정밀도를 있다고 오해하게 된다 — 7장의 위험도 회수, 12-5 의 IP·시각 제거와 같은 방향
중복을 피했다	그룹 키를 Java 에 다시 구현하지 않고 <code>source_doc_id</code> 로 찾는다. 9-7 에서 포트 번호로 같은 실수를 이미 한 번 했다
유지	9-8 의 관문(인증·쿼터·동시 실행)은 그대로다 — 해설이 없는 사건을 누를 때 여전히 필요하다

A-8. 판 관리 규칙

- 개선이 반영되면 **새 판**을 만든다. 기존 판은 `docs-versions/` 에 그대로 보존한다
- 사실이 바뀐 곳은 새 판에서 **반드시 갱신**한다 — 갱신하지 않으면 문서가 스스로 모순된다
- 이전 판의 진단이 틀린 것으로 밝혀지면, 지우지 말고 **무엇이 왜 틀렸는지**를 새 판에 남긴다